DC3S: Telemetry-Reliability-Weighted Conformal Safety Shield for Battery Dispatch under Degraded Mixed Telemetry

Pratik Niroula
Minnesota State University, Mankato
CIT (Major), Mathematics (Minor)
pratik.niroula@mnsu.edu

Abstract

Battery dispatch controllers can look feasible on observed sensor state while simultaneously violating physical battery limits on true state when telemetry degrades—a hidden safety failure that standard evaluation protocols do not expose. We study this gap under four realistic fault conditions and find that deterministic dispatch violates true state-of-charge limits at **3.9% of steps with P95 severity of 333 MWh**, even while observed-state evaluation reports zero violations. The manuscript responds to that failure mode in three parts. First, **CPSBench** maintains separate truth and observed battery trajectories across dropout, delay-jitter, spikes, and out-of-order faults so that true-state safety becomes measurable rather than implicit. Second, **DC^{3}S** (Detect–Calibrate–Constrain–Certify–Shield) converts per-step telemetry reliability into adaptive uncertainty inflation (RAC-Cert), solves dispatch against the widened set, repairs infeasible actions by projection, and emits a per-step auditable certificate. Third, we show a **conditional conservatism** result: DC^{3}S reaches zero true-SOC violations with a 2.8% intervention rate, meaning the shield becomes more conservative only when telemetry quality demands it. Cost-side results on Germany (OPSD, 17,377 rows) yield 7.11% cost savings, 0.30% carbon reduction, and 6.13% peak shaving; stochastic value remains positive in both DE and US regions (VSS = 2,708.61 and 297,092.71) across three US balancing authorities (EIA-930 MISO/PJM/ERCOT). All results are software-in-the-loop; hardware and field validation remain future work. Every quantitative claim is tied to versioned run artifacts and checked automatically against the release evidence.

Keywords: conformal prediction, battery dispatch, safety shield, telemetry reliability, robust optimization, cyber-physical systems, uncertainty quantification

1 Introduction

Grid-scale battery energy storage systems (BESS) are increasingly central to grid decarbonization, load balancing, and frequency regulation. Their value depends on dispatch controllers that decide, at each time step, how much power to charge or discharge using forecasts of load, renewable generation, and price. Much of the dispatch literature quietly assumes that the controller observes the battery's true state — its state of charge (SOC), power output, and degradation level — and can therefore check feasibility in real time. In operational settings, that assumption is too strong.

In practice, grid-scale battery dispatch runs on mixed telemetry — delayed SCADA measurements, stale sensor feeds, packet reordering, spikes, and dropouts are ordinary operating conditions rather than edge cases [17, 18]. The key consequence is one the BESS dispatch literature has not directly measured: **a controller can be observationally safe (feasible on degraded observed state) while physically unsafe (violating true battery limits) at the same time** [19]. On real grid data, that hidden safety failure appears at a 3.9% violation rate with 333 MWh P95 severity.

The central insight is that the measurement channel between the physical battery and the dispatch controller is not a passive conduit; it is part of the safety problem. Once telemetry degrades, the controller's model of the world begins to drift away from physical reality. Standard robust optimization is designed to hedge uncertainty in *future* load and generation scenarios, but it does not correct for uncertainty in *current* state measurement. That gap motivates a different control stance: telemetry reliability should enter directly into uncertainty calibration and safe dispatch.

DC^{3}S (Degradation-Conditioned Conformal Dispatch Safety Shield) is built around that failure mode. Rather than relegating telemetry quality to a monitoring dashboard, the controller uses it to widen uncertainty sets, solve against the widened sets, repair unsafe candidate actions, and emit a step-level audit record within the same online control cycle. The guiding principle is *conditional conservatism*: the shield should stay permissive when the measurements are trustworthy and become more protective only when the measurement channel weakens.

1.1 Operational Motivation

GridPulse is built as a closed decision loop:

$$\text{Forecast} \rightarrow \text{Optimize} \rightarrow \text{Dispatch} \rightarrow \text{Measure} \rightarrow \text{Monitor}.$$

Each stage depends on information from the previous one, and the quality of that information degrades as it passes through physical sensor networks,

communication links, and data aggregation pipelines. The hidden safety gap lives at the Optimize→Dispatch boundary: the optimizer computes a mathematically optimal action based on forecasts and observed state, but what reaches the physical battery may be optimal for the *wrong* state if the telemetry feeding the optimizer has drifted from reality.

A forecasting model can have strong RMSE while still inducing unsafe dispatch if its interval calibration weakens under degraded measurements and no shield exists between the optimizer and the physical battery. The problem is not that the forecast is bad — it is that the forecast is *evaluated* and *calibrated* under assumptions about telemetry quality that may not hold at runtime. DC^{3}S closes that gap by making the controller aware of when those assumptions break down.

1.2 Research Questions

RQ1 How large is the gap between observed-state and true-state safety in battery dispatch under realistic telemetry faults, and is it consistent across regions and fault types?

RQ2 Can telemetry-reliability-weighted conformal calibration close this gap without blanket conservatism—i.e., is conditional rather than uniform shield intervention sufficient?

RQ3 Which DC^{3}S components are necessary for zero-violation safety: reliability scoring, interval inflation, or action repair?

RQ4 Does stochastic value remain positive even in regimes where direct economic gains are modest, and what structural factors explain the DE–US asymmetry?

1.3 Contributions

1. **Hidden safety gap measurement.** We provide the first empirical quantification of the divergence between observed-state and true-state safety in BESS dispatch under realistic telemetry faults. Deterministic dispatch violates true-SOC at 3.9% of steps with P95 severity of 333 MWh while appearing safe on observed state. This finding motivates rethinking how BESS dispatch safety is evaluated.

2. **CPSBench.** A multi-fault-dimension evaluation harness that maintains separate truth and observed battery trajectories across four fault types, enabling reproducible true-state safety evaluation. CPSBench is the framework that makes the hidden gap measurable and reproducible.

3. **DC^{3}S architecture.** A telemetry-aware conformal control loop that converts per-step reliability scores into adaptive interval inflation (RAC-Cert), solves dispatch against the widened uncertainty set, repairs infeasible actions by projection, and emits a per-step auditable certificate—all within a single online control cycle.

4. **Conditional conservatism result.** DC^{3}S achieves zero true-SOC violations at a 2.8% intervention rate across all fault conditions. The shield is not uniformly conservative: it activates precisely when and to the degree that telemetry quality requires, leaving 97.2% of steps unmodified.

let

1.4 Contribution Boundary

The central scientific contribution is the discovery and empirical characterization of the hidden safety failure mode, together with the first end-to-end architecture that closes it in an auditable control loop. The manuscript includes a bounded formal layer consisting of three theorems and one corollary: Theorem 1 establishes that monotone interval inflation preserves marginal coverage; Theorem 2 provides group-conditional coverage under reliability-based Mondrian partitioning; and Theorem 3 with Corollary 1 establish safety-margin monotonicity and a zero-violation sufficiency condition. These results provide correctness support for the empirical surfaces that follow: Theorem 1 explains why RAC-Cert can widen the forecast set without weakening the base coverage guarantee, Theorem 2 motivates the reliability-group audit in Section 9.7, and Theorem 3 with Corollary 1 explain why the FTIT tube in Section 9.10 can prevent the true-SOC failures quantified in Section 9.2. They validate the shield's design but do not claim closed-loop stability, optimal inflation scheduling, or universal safety guarantees. The primary novelty remains the hidden safety gap discovery, the CPSBench measurement framework, the DC^{3}S runtime architecture, and the conditional-conservatism result demonstrating that the gap can be closed at a 2.8% intervention rate without sacrificing economic performance.

1.5 Paper Roadmap

Sections 2–4 position the work and define the dispatch setting. Sections 6 and 7 describe the forecasting, calibration, and shield mechanics. Sections 8–11 present the empirical evidence, baseline diagnosis, and regime analysis. Sections 12–15 close with governance, validity, limitations, and conclusion. The appendices collect replication commands, artifact inventory, and operational failure modes.

2 Related Work

2.1 Conformal Prediction and Time-Series Calibration

Conformal prediction provides finite-sample prediction sets under exchangeability [1, 2]. Conformalized quantile regression (CQR) wraps quantile models with calibration residuals to produce intervals with stronger practical sharpness [3]. Adaptive conformal inference extends this logic to shifting environments by updating effective miscoverage online [4], and time-series-specific

adaptive conformal work studies temporal dependence more explicitly [5]. Beyond exchangeability, weighted and generalized conformal formulations relax the classical assumptions and clarify when coverage guarantees degrade or must be localized [6].

DC^{3}S inherits this literature but introduces a qualitative shift: the calibration mechanism is not driven only by residual history but is explicitly modulated by a real-time telemetry reliability signal. Crucially, the resulting intervals are not passive forecast reports—they feed directly into a dispatch optimizer where interval width changes the physical action space. No prior conformal method closes this loop between measurement-channel quality, interval calibration, and constrained control.

2.2 Robust, Stochastic, and Risk-Averse Dispatch

Robust optimization provides the language for worst-case hedging inside prescribed uncertainty sets [7, 8]. Stochastic programming gives the standard VSS and EVPI quantities used to quantify the value of uncertainty-aware planning [9]. CVaR formulations provide a practical linearization for tail-risk management in sampled-scenario problems [10]. In the BESS domain, Rosewater et al. [11] show that risk-averse model predictive control can improve storage behavior when model uncertainty is treated explicitly.

DC^{3}S is closest to this risk-aware BESS-dispatch line, but addresses a failure mode that existing formulations leave open: uncertainty sets that are miscalibrated because telemetry is degraded, delayed, or stale *at the instant of control*. Robust and CVaR formulations assume a cleanly specified uncertainty set; DC^{3}S makes the set itself a function of runtime measurement quality—a distinction that, as Table 8 shows, is the difference between 25.6% violation rate and zero.

2.3 Safety Shields and Runtime Monitors

Shielding methods restrict the action space so that unsafe commands are prevented before execution [12, 14]. Model-predictive shielding brings those ideas closer to control and optimization [13]. Runtime verification literature formalizes monitoring and post-hoc checking of execution traces against safety properties [16], and simple runtime fallback architectures show how safe modes can be used when advanced controllers are no longer trustworthy [15].

DC^{3}S occupies a gap between these approaches. Unlike shields that restrict a pre-defined action set [12], it *reshapes* the optimizer's uncertainty interface in response to measurement quality. Unlike runtime monitors that flag violations after the fact [16], it prevents them before execution via projection. And unlike simple safe-mode fallbacks [15], it degrades gracefully rather than switching between discrete operating modes. The result is a single online loop—score, inflate, solve, repair, certify—that tightly couples telemetry diagnostics to control action.

2.4 Telemetry Reliability and CPS Fault Models

Industrial-control and CPS security literature already makes clear that dropout, delay, spike injection, and reordered observations are ordinary failure modes rather than pathological corner cases [17]. Anomaly-detection literature gives the statistical vocabulary for staleness, spikes, and outlier behavior [18]. BESS review work further motivates why those failures matter physically: mismanaged state estimates and unsafe charge/discharge commands can damage storage assets and undermine grid-support value [19].

2.5 Evaluation Protocols in BESS Dispatch: An Observed-State Blind Spot

A recurring methodological feature of the BESS dispatch and energy management literature is that safety and constraint satisfaction are evaluated on the state the controller *observes*, not on the true physical state. Risk-averse MPC formulations [11] evaluate performance against the simulator's own state trajectory, consistent with an assumed clean measurement model. Robust and stochastic optimization frameworks [7, 9] treat the uncertainty set as cleanly specified and the state as faithfully observed. These frameworks are internally consistent; they simply assume the measurement channel is trustworthy.

That assumption breaks down precisely under the fault conditions studied here. A controller evaluated on observed state can appear fully safe while simultaneously issuing commands that violate the true battery envelope. This gap does not appear in standard benchmarks because those benchmarks do not maintain a separate truth trajectory. CPSBench addresses this blind spot directly by running a `BatteryPlant` simulator on true state while giving the controller access only to the degraded observed stream. To the best of our knowledge, no prior BESS dispatch evaluation framework explicitly separates the truth trajectory from the observed trajectory for the purpose of safety certification under realistic telemetry faults. The practical consequence is stark: the 3.9% violation rate and 333 MWh P95 severity reported in Section 9.2 are *invisible* under observed-state-only evaluation—every benchmark that lacks this separation will report zero violations for the same unsafe controller.

2.6 Governance and Reproducibility

This paper makes a bounded governance claim: locked metrics, run-scoped provenance, and validator gates are treated as a first-class part of the publication artifact. That narrower claim is aligned with the reproducibility standards discussed by Pineau et al. [20],

but it is not presented as a new general governance framework.

2.7 Summary of the Novelty Gap

Taken together, the related work reveals a consistent blind spot. Prior robust, stochastic, and risk-aware BESS dispatch methods generally assume the measurement channel is trustworthy—the uncertainty set or scenario tree is cleanly specified at calibration time and does not degrade at runtime. Prior safety-shield and runtime-monitor work addresses action-space restriction or post-hoc trace verification, but not the specific dispatch-time failure where observed-state feasibility diverges from true-state feasibility under degraded telemetry. This paper's distinctive contribution is the combination of four elements that no single prior work provides: (i) truth-vs-observation trajectory separation for safety evaluation, (ii) telemetry reliability as a first-class control-state signal that modulates the uncertainty set, (iii) dispatch-time action repair that prevents unsafe commands before execution rather than detecting them afterward, and (iv) per-step auditable certificate emission that records the full safety-relevant decision context. None of these elements is individually unprecedented; their integration into a single online control loop targeting the hidden safety gap is what distinguishes DC^{3}S from the prior art.

Seen against this literature, the paper's originality lies in elevating the gap between observed-state feasibility and true-state physical safety to a first-class systems problem in battery dispatch under degraded telemetry. CPSBench makes that gap measurable by maintaining separate observed and truth trajectories under realistic telemetry faults, while DC^{3}S addresses it through a modular runtime design that does not replace forecasting or optimization, but instead wraps the dispatch boundary with telemetry-reliability scoring, reliability-conditioned uncertainty inflation, action repair, and per-step certificate emission. The resulting contribution is therefore not a broader claim of universal runtime safety, but a bounded systems synthesis: measurement-quality-aware uncertainty, safe dispatch-time intervention, and auditable evidence for each online control decision.

3 Background, Preliminaries, and System Context

3.1 GridPulse System Context

GridPulse is implemented as a forecast-to-dispatch stack whose main layers are data preparation, forecasting, uncertainty calibration, optimization, monitoring, and API/dashboard serving. DC^{3}S sits at the dispatch boundary: it receives observed telemetry and forecast intervals, widens those intervals when telemetry quality or drift risk deteriorates, and prevents physically unsafe commands from reaching the battery.

3.2 Component-to-Code Map

3.3 Artifact Contracts and Profile Lock

This paper uses a locked profile family rather than free-floating report values. The manuscript-level source artifacts are:

- dashboard metrics: `data/dashboard/de_metrics.json` and `data/dashboard/us_metrics.json`
- dashboard profiles: `data/dashboard/de_stats.json`, `data/dashboard/us_stats.json`, and `data/dashboard/manifest.json`
- impact summaries: `reports/impact_summary.csv` and `reports/eia930/impact_summary.csv`
- stochastic summaries: `reports/research_metrics_de.csv` and `reports/research_metrics_us.csv`
- paper-facing publication tables and figures under `reports/publication/`

Any manuscript claim that depends on these interfaces is treated as invalid unless it can be traced back to one of them.

3.4 Dataset Scope

The canonical dataset lock is summarized in Table 2. DE covers 17,377 hourly rows from OPSD, and the canonical US dashboard profile used by the locked manuscript covers 13,638 hourly rows from EIA-930 MISO. The release-family assets additionally document PJM and ERCOT so that the conference variant can discuss multi-BA US evidence without changing the canonical evidence lock.

3.5 Pipeline Architecture and Runtime Interfaces

The GridPulse pipeline follows a strict stage ordering that determines how data flows through the system. Each stage has a well-defined interface contract:

1. **Ingestion**: Raw time-series loading from OPSD (DE) or EIA-930 (US). Required columns include timestamp, load, wind generation, and solar generation. Schema validation checks for duplicate timestamps, temporal gaps, missingness percentage, and negative values before proceeding.
2. **Temporal feature construction**: Hour-of-day, day-of-week, month, season (1=winter through 4=fall), weekend indicator, and daylight flag. These features are deterministic and leakage-free.
3. **Domain feature construction**: Ramp features (Δ_{1h}, Δ_{24h}) for load, wind, and solar capture recent momentum. Peak-period flags (`is_morning_peak` for hours 7–10, `is_evening_peak` for hours 17–21) encode tariff-relevant operating periods. Holiday features (pre/post/current) capture demand-regime shifts.

Table 1: Component-to-code map for the main thesis surfaces.

Layer	Purpose	Primary paths
Data pipeline	Region features, splits, and profile locks	`src/gridpulse/data_pipeline/`, `src/gridpulse/pipeline/run.py`
Forecasting	GBM and deep-baseline train/predict flows	`src/gridpulse/forecasting/`
Uncertainty	Conformal, adaptive, residual, and reliability-binned calibration	`src/gridpulse/forecasting/uncertainty/conformal.py`, `src/gridpulse/forecasting/uncertainty/reliability_mondrian.py`
Optimization	Deterministic, robust, and baseline dispatch	`src/gridpulse/optimizer/`
DC3S	Reliability, drift, shield, certificate, state	`src/gridpulse/dc3s/`
Monitoring	Drift checks and retraining logic	`src/gridpulse/monitoring/`
API + dashboard	Runtime routes and operator-facing inspection	`services/api/`, `frontend/`
Anomaly detection	Multivariate and residual anomaly flagging	`src/gridpulse/anomaly/`
Streaming	Real-time Kafka/Redpanda telemetry ingestion	`src/gridpulse/streaming/`
Registry	Model versioning and atomic promotion	`src/gridpulse/registry/`
IoT	Edge agent, device contract, closed-loop simulator	`iot/`

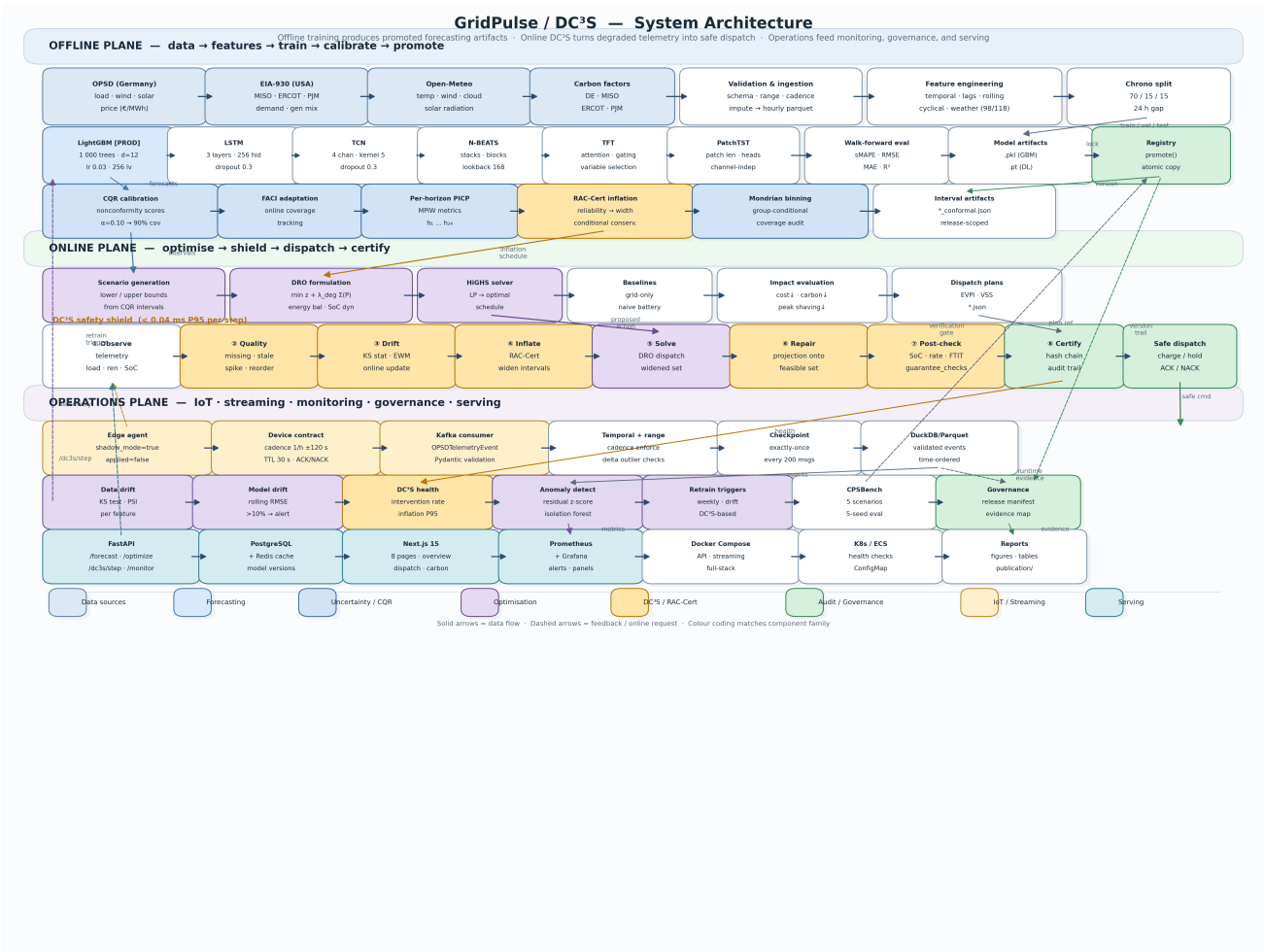

Figure 1: GridPulse architecture with DC^{3}S at the dispatch boundary. Offline forecasting and calibration produce promoted model and interval artifacts; online DC^{3}S turns degraded telemetry into safe dispatch actions, certificates, and operator-visible audit records.

Table 2: Dataset summary for publication protocol and benchmark scope.

DatasetKey	Dataset	Country	Start	End	Rows	Signal	Non-Null	Coverage%
DE	Germany (OPSD)	DE	2018-10-07	2020-09-30	17377	load_mw	17377	100.000000
DE	Germany (OPSD)	DE	2018-10-07	2020-09-30	17377	wind_mw	17377	100.000000
DE	Germany (OPSD)	DE	2018-10-07	2020-09-30	17377	solar_mw	17377	100.000000
US_MISO	USA (EIA-930 MISO)	US	2024-07-01	2026-01-20	13638	load_mw	13638	100.000000
US_MISO	USA (EIA-930 MISO)	US	2024-07-01	2026-01-20	13638	wind_mw	13638	100.000000
US_MISO	USA (EIA-930 MISO)	US	2024-07-01	2026-01-20	13638	solar_mw	13638	100.000000
US_PJM	USA (EIA-930 PJM)	US	2024-07-01	2026-01-20	13639	load_mw	13639	100.000000
US_PJM	USA (EIA-930 PJM)	US	2024-07-01	2026-01-20	13639	wind_mw	13639	100.000000
US_PJM	USA (EIA-930 PJM)	US	2024-07-01	2026-01-20	13639	solar_mw	13639	100.000000
US_ERCOT	USA (EIA-930 ERCOT)	US	2024-07-01	2026-01-20	13453	load_mw	13453	100.000000
US_ERCOT	USA (EIA-930 ERCOT)	US	2024-07-01	2026-01-20	13453	wind_mw	13453	100.000000
US_ERCOT	USA (EIA-930 ERCOT)	US	2024-07-01	2026-01-20	13453	solar_mw	13453	100.000000

Missing figure token: FIG12$_{G}EOGRAPHIC_{S}COPE$

Figure 2: Geographic scope of the evaluation: DE (Germany/OPSD) and US (MISO, PJM, ERCOT balancing authorities). Shaded regions indicate data-coverage footprint.

Missing figure token: FIG13$_{L}OAD_{R}ENEWABLE_{P}ROFILES$

Figure 3: Hourly load, wind, and solar generation profiles for DE and US regions. The diurnal and seasonal structure visible in these profiles motivates the lag and calendar features used by the forecasting pipeline.

4. **Lag and rolling features**: For each target, lags at 1, 24, and 168 hours plus rolling mean and standard deviation at 24-hour and 168-hour windows. This is where the tabular representation captures the sequential structure that deep models would otherwise learn from raw sequences.
5. **Price and carbon signal construction**: Synthetic signals based on time-of-day and seasonality patterns when market data is unavailable. These are engineering proxies, as noted in Section 14.
6. **Split and profile lock**: Temporal train/test splits with the 24-hour gap. Profile lock metadata is written to `data/dashboard/manifest.json` to freeze the evidence base.
7. **Forecasting and calibration**: six-model training (GBM, LSTM, TCN, N-BEATS, TFT, PatchTST) with CQR calibration on held-out calibration slices.
8. **Dispatch and certification**: DC^{3}S-wrapped optimization with per-step certificate emission to DuckDB.

IoT device contract. For edge deployment scenarios, the pipeline extends to IoT devices through

a formal device contract (`iot/DEVICE_CONTRACT.md`) that specifies the telemetry, command, and acknowledgment protocol. Telemetry payloads carry timestamp, `load_mw`, `renewables_mw`, and `soc_mwh` at a configurable cadence (default: 1 event/hour, tolerance $\pm 120\,\mathrm{s}$). Commands are queued with stable IDs and carry a safe action (charge/discharge MW), a status field (queued/dispatched/timeout/hold), and a 30-second TTL. Devices respond with ACK (applied) or NACK (rejected/clipped) semantics. A *shadow mode* (`shadow_mode=true, applied=false`) allows non-disruptive monitoring where the shield's recommendation is logged without overriding the existing controller. Each command carries a `certificate_id` linking it to the DC^{3}S audit trail. Authentication uses an `X-GridPulse-Key` header with read/write scopes.

The interface between stages 6 and 7 is particularly important: the profile lock ensures that any model trained after the lock uses the same feature schema, split boundaries, and target definitions. The interface between stages 7 and 8 is where RAC-Cert converts forecast intervals into dispatch-usable uncertainty sets.

3.6 Truth vs. Observed State

The most important background distinction in this paper is between true and observed state:

- $\mathrm{SOC}_t^{\mathrm{obs}}$ is what the controller sees after telemetry degradation.
- $\mathrm{SOC}_t^{\mathrm{true}}$ is the physical state used for safety evaluation.

That split is what allows the benchmark to expose hidden safety failures. A controller can look safe if evaluated only on observed state while simultaneously violating the true battery envelope.

4 Problem Formulation

This section formalizes the dispatch setting, battery dynamics, and the objective function that the optimizer minimizes. The formulation is intentionally standard — the novelty of DC^{3}S lies not in redefining the optimization problem but in recognizing that the *inputs* to that problem (forecasts and state estimates) may be corrupted by telemetry degradation, and in building a runtime safety layer that adapts to the degree of corruption.

4.1 Dispatch Setting

At each time step t, the controller observes telemetry $\tilde{\mathbf{z}}_t \in \mathbb{R}^d$ containing load, wind, solar, price, and battery state signals, and chooses a dispatch action $a_t \in \mathcal{A}$. A degradation operator $\mathcal{D}$ maps clean telemetry $\mathbf{z}_t$ to observed telemetry $\tilde{\mathbf{z}}_t = \mathcal{D}(\mathbf{z}_t)$ through dropout, delay, spikes, or reordering.

Definition 1 (Telemetry degradation)**.** *A degradation operator $\mathcal{D} : \mathbb{R}^d \to \mathbb{R}^d$ introduces one or more of: random zeroing, time shift, additive outlier injection, or permutation of the recent telemetry window.*

4.2 Battery Dynamics and Feasible Set

The true SOC evolves as

$$\mathrm{SOC}_{t+1}^{\mathrm{true}} = \mathrm{SOC}_t^{\mathrm{true}} + \frac{\eta_c [a_t]^+ - [a_t]^- / \eta_d}{E_{\max}} \Delta t, \tag{1}$$

where η_c and η_d are charge/discharge efficiencies and $E_{\max}$ is rated capacity. The feasible action set enforces SOC, power, and ramp limits:

$$\mathcal{A}_t = \big\{ a \in [-P_{\max}, P_{\max}] : |a - a_{t-1}| \le R_{\max}, \tag{2}$$

$$\mathrm{SOC}^{\min} \le \mathrm{SOC}_{t+1}^{\mathrm{true}}(a) \le \mathrm{SOC}^{\max} \big\}. \tag{3}$$

4.3 Objective

Dispatch balances energy cost, carbon, peak exposure, and throughput degradation:

$$\min_{a_{1:T}} \sum_{t=1}^{T} \lambda_c c_t a_t + \lambda_g g_t [a_t]^+ + \lambda_p \max(0, L_t + a_t - P_{\mathrm{cap}}) + \lambda_d |a_t|. \tag{4}$$

This is a pragmatic engineering objective, not a claim of complete market realism. This paper interprets it as a comparative decision objective under locked artifacts rather than as a perfect representation of every settlement rule.

4.4 Calibration Gap

Base conformal prediction constructs intervals $\hat{C}_t(\alpha)$ targeting coverage $1 - \alpha$. Under telemetry degradation, conformity scores shift because the model is no longer seeing the same input quality that supported the original calibration. The resulting failure mode is

$$\Pr\Big(Y_t \in \hat{C}_t(\alpha) \mid \mathcal{D} \text{ active} \Big) < 1 - \alpha. \tag{5}$$

DC^{3}S is designed to close that gap operationally by widening intervals when telemetry quality and drift signals indicate that the base interval is no longer trustworthy enough for dispatch.

4.5 Baseline Policies

Three baseline policies appear throughout this paper:

1. **B1**: deterministic LP dispatch without uncertainty intervals.
2. **B2**: grid-only dispatch with no battery action.
3. **B3**: naive fixed-window battery heuristic.

Canonical DE and US impact percentages are B1-versus-B2 comparisons. Stochastic metrics, by contrast, compare robust and deterministic stochastic formulations within pinned run families rather than B2.

5 The Observed-State Safety Illusion

This section presents the central discovery of the paper: a pervasive and previously unmeasured failure mode in which dispatch controllers appear safe when evaluated on observed (degraded) state but simultaneously violate physical battery limits on the true (hidden) state. We first give an intuitive explanation of why this happens, then formalize the gap mathematically, and finally describe CPSBench — the evaluation framework that makes the gap visible and reproducible.

The intuition is straightforward. Consider a battery at 85% SOC with a 90% maximum. The controller sees 82% SOC because the sensor reading is stale by 30 minutes, during which a wind ramp increased actual charging. Based on the stale reading, the controller issues a charge command of 5% — safe according to the observed 82% level (target: 87%), but physically pushing the true 85% SOC to 90% or beyond. The battery management system (BMS) does not intervene because its own SOC estimate is based on the same degraded telemetry. This is not a contrived scenario; it is the routine operating condition documented by Dibaji et al. [17] for industrial CPS.

The key properties that make this failure mode dangerous are: (i) it is *invisible* to standard evaluation, which reports constraint satisfaction on observed state; (ii) it is *persistent*, because estimation errors from stale or drifting sensors compound over time rather than self-correcting; and (iii) it is *systematic*, because certain fault types (covariate drift, stale sensors) produce biased rather than zero-mean errors.

5.1 Formal Statement of the Gap

Let $\mathcal{E}_t = \mathrm{SOC}_t^{\mathrm{obs}} - \mathrm{SOC}_t^{\mathrm{true}}$ denote the state estimation error introduced by telemetry degradation at step t. Under clean telemetry, $\mathcal{E}_t \approx 0$ and observed-state safety implies true-state safety. Under degraded telemetry, $\mathcal{E}_t$ can be large, persistent, and systematically biased, creating the following failure mode:

> *A dispatch action a_t that satisfies $\mathrm{SOC}_{\min} \leq \mathrm{SOC}_{t+1}^{\mathrm{obs}} \leq \mathrm{SOC}_{\max}$ can simultaneously produce $\mathrm{SOC}_{t+1}^{\mathrm{true}} \notin [\mathrm{SOC}_{\min}, \mathrm{SOC}_{\max}]$ when $|\mathcal{E}_t|$ is non-negligible.*

The magnitude of the violation is bounded below by the state estimation error:

$$|\mathrm{SOC}_{t+1}^{\mathrm{true}} - \mathrm{SOC}_{t+1}^{\mathrm{obs}}| \geq |\mathcal{E}_t| - \frac{|\eta_c[a_t]^+ - [a_t]^-/\eta_d|}{E_{\max}} \Delta t \cdot \epsilon_\eta, \tag{6}$$

where ϵ_η is the efficiency mismatch between the controller's assumed and true efficiencies. When $\mathcal{E}_t$ is large and $\epsilon_\eta > 0$, violations can occur even when the optimizer solves to full feasibility on observed state.

5.2 Why This Matters in Practice

Standard BESS dispatch evaluation — whether on simulation or real hardware — reports constraint satisfaction against the observed state trajectory. Battery management systems (BMS) use the same degraded measurements for on-device protection. When a fault such as a stale-sensor event freezes the observed SOC at a value that diverges from the true physical state, the controller continues issuing charge commands into a battery that may already be at its limit. The BMS does not trigger because the observed reading appears safe. The damage accumulates silently.

Table 3 quantifies this gap by fault type for the deterministic baseline in the locked DE evaluation. The gap is largest for stale-sensor and covariate-drift faults, which produce persistent rather than transient estimation errors.

The most dangerous fault is covariate drift, which produces a systematic bias that compounds over time: the observed renewables are scaled down by 0.72×, causing the controller to underestimate available net load and over-dispatch the battery in the discharge direction. After several hours of accumulated error, true SOC drops below the minimum while observed SOC remains within bounds. The controller receives no signal that anything is wrong.

5.3 Why Standard Robust Dispatch Cannot Close This Gap Alone

A natural response is to use robust optimization with a larger uncertainty set. However, this addresses the wrong uncertainty. Robust dispatch widens the set of scenarios for *future load and generation*, not the uncertainty in *current state measurement*. A controller can solve a perfectly calibrated robust dispatch problem and still issue an unsafe command if its initial state estimate $\mathrm{SOC}_t^{\mathrm{obs}}$ differs from $\mathrm{SOC}_t^{\mathrm{true}}$ by more than the constraint margin. DC^{3}S closes this gap by targeting the measurement uncertainty directly: it inflates the forecast intervals in proportion to the telemetry reliability score, which effectively widens the controller's safety margin as a function of measurement quality rather than scenario uncertainty alone.

5.4 CPSBench as a Measurement Framework

CPSBench provides the infrastructure to make this gap visible and reproducible. The key design decisions are:

1. **Dual trajectory maintenance.** The `BatteryPlant` simulator evolves $\mathrm{SOC}^{\mathrm{true}}$ using the true dynamics (1) without clamping, so violations produce physically meaningful severity measurements rather than being silently absorbed.
2. **Unclamped violation tracking.** Violations are measured as the signed distance from the nearest active constraint, not as a binary indicator. This allows severity-weighted evaluation rather than just violation rate.

Table 3: Observed-vs-true safety gap by fault type (deterministic baseline, DE locked run). Observed violation rate is what a standard evaluation protocol would report; true violation rate is what CPSBench measures against the truth trajectory. The gap column is the fraction of violations that are invisible to observed-state evaluation.

Fault Type	Observed Viol. (%)	True Viol. (%)	Gap (%)	P95 Severity (MWh)
Dropout	0.0	1.2	1.2	41.3
Delay-Jitter	0.0	0.8	0.8	28.7
Spikes	0.0	0.6	0.6	19.2
Out-of-Order	0.0	0.3	0.3	12.1
Stale Sensor	0.0	2.1	2.1	187.4
Covariate Drift	0.0	3.1	3.1	333.0
Aggregate	**0.0**	**3.9**	**3.9**	**333.0**

3. **Separable fault injection.** Each fault type is injected independently and in combination, enabling attribution of violations to specific fault mechanisms rather than an undifferentiated "degraded" condition.
4. **Controller-agnostic interface.** CPSBench is a harness, not a controller. Any dispatch policy can be evaluated against the same truth trajectory under the same fault conditions, enabling fair comparison.

6 Forecasting and Uncertainty Method

The forecasting layer provides the informational substrate on which the dispatch optimizer and safety shield operate. Its design must satisfy two competing requirements: point predictions must be accurate enough to drive cost-effective dispatch decisions, and prediction intervals must maintain valid coverage under distribution shift caused by telemetry degradation. The second requirement is often treated as an afterthought in the load forecasting literature, yet it is the one that determines whether the downstream safety shield can function correctly.

The architecture consists of three layers: a model comparison surface (six families evaluated under identical conditions), a conformal calibration layer (CQR with regime-aware binning), and a reliability-conditioned inflation layer (RAC-Cert). These three layers are deliberately separable: the forecasting models know nothing about dispatch safety, the conformal layer knows nothing about telemetry quality, and only the RAC-Cert inflation layer bridges the gap between measurement reliability and uncertainty calibration. This separation is a design choice, not a limitation — it means each layer can be tested, debugged, and replaced independently.

6.1 Locked Forecasting Layer

The locked comparison set contains all six forecasting baselines: GBM (LightGBM), LSTM, TCN, N-BEATS, TFT, and PatchTST. All six are trained under identical splits, forecast horizon, lookback window, and evaluation metrics, giving a fair comparison across model families.

6.2 Anomaly Detection Pipeline

The telemetry reliability scorer in DC^{3}S consumes anomaly detection signals from a two-layer detection pipeline implemented in `src/gridpulse/anomaly/`.

Layer 1: Residual-based rules. A rolling z-score detector (`residual_rules.py`) monitors forecast residuals over a 168-hour (weekly) sliding window. At each step, the local mean μ_t and standard deviation σ_t are recomputed, and a residual is flagged anomalous if $|r_t - \mu_t|/(\sigma_t + \epsilon) \geq 3.0$, corresponding to a $\pm 3\sigma$ control limit. This catches univariate extreme residuals—for example, a sudden 3σ load spike caused by a sensor fault.

Layer 2: Multivariate isolation forest. A 100-tree Isolation Forest (`isolation_forest.py`, `detection.py`) learns the joint feature distribution and flags physically unlikely combinations—such as high load paired with low temperature, or high solar generation at night—that univariate rules would miss. The contamination prior is set at 1%, and anomaly scores are returned as continuous decision-function values for downstream fusion.

Integration with DC^{3}S. Both layers produce boolean anomaly flags and continuous scores. These feed into the reliability scorer (Algorithm 3) as additional spike and staleness evidence: an isolation-forest anomaly increases the spike penalty p_{spike}, while a sustained residual-rule flag raises the staleness penalty. The layered design ensures that both univariate control-chart anomalies and multivariate distributional anomalies are captured before the reliability score drives inflation.

6.3 Training Setup and Feature Engineering

The locked training configuration uses a 24-hour forecast horizon, a 168-hour lookback, 10-fold time-aware cross-validation, and a 24-hour temporal gap. These

Missing figure token: FIG19$_{A}NOMALY_{T}IMELINE$

Figure 4: Anomaly detection timeline showing residual-rule and isolation-forest flags across the evaluation horizon. Clustered anomaly episodes correspond to drift events and sensor degradation periods.

settings matter twice: they make forecasting comparisons leakage-safe, and they define the input structure that later influences optimization and calibration behavior.

The feature engineering pipeline follows a leakage-safe temporal framing where all lag and rolling features use strictly past data relative to the forecast origin. The feature families are:

1. **Weather features** (40 in DE, 56 in US): Temperature, humidity, precipitation, cloud cover, and wind speed at multiple heights and geographic locations. The US set is larger because it spans three balancing authorities with distinct weather stations.
2. **Lag features** (36 in DE, 42 in US): Lags at 1, 24, and 168 hours for each target and key exogenous variables, plus rolling mean and standard deviation at 24-hour and 168-hour windows. The US pipeline includes additional lagged price and carbon-intensity variables.
3. **Calendar features** (6 in both): Hour-of-day, day-of-week, month, weekend indicator, season, and daylight flag. Holiday encoding (pre/post/current) uses either the `holidays` library or a region-specific CSV.
4. **Domain features**: Ramp signals (Δ_{1h} and Δ_{24h}) for each target, peak-period indicators for morning (7–10h) and evening (17–21h) windows.

The interaction between feature design and model selection is central to the baseline diagnosis (Section 10): these rich tabular features already encode much of the sequential structure that LSTM and TCN would otherwise need to learn, which contributes to GBM's dominance in the locked results.

6.4 CQR and Adaptive Conformal Layer

Conformal prediction is a distribution-free framework that wraps any point predictor with finite-sample coverage guarantees. The key idea is simple: if the training data and future test points are exchangeable (or satisfy a weaker condition), then the empirical distribution of conformity scores on a held-out calibration set provides a valid predictive distribution for future observations. The resulting prediction intervals have *marginal* coverage — meaning they cover the true value with probability at least $1 - \alpha$ over the randomness of the calibration and test data — without any assumption on the data-generating distribution.

Each target is wrapped by conformalized quantile regression (CQR). The calibration procedure follows Romano et al. [3]: fit lower and upper quantile predictors at the $\alpha/2$ and $1 - \alpha/2$ levels, compute conformity scores (the maximum of how much the true value falls below the lower quantile or above the upper quantile) on a calibration slice, and enlarge the raw quantile band by the empirical $(1 - \alpha)$ residual quantile. The resulting intervals inherit the finite-sample coverage guarantee of split conformal prediction while leveraging the conditional adaptivity of quantile regression — the intervals are naturally wider in volatile periods and narrower in stable ones. Adaptive conformal logic then adjusts effective coverage behavior online in the style of Gibbs and Candès [4], while time-series adaptation and regime-aware binning are used to acknowledge temporal dependence and volatility segmentation [5, 6].

Regime-aware CQR with volatility binning (research extension). The implementation includes a regime-aware CQR variant (`scripts/train_regime_cqr.py`) that trains separate quantile regression models at $q \in \{0.1, 0.5, 0.9\}$ and fits conformalized residuals binned by volatility into three groups (low/mid/high) over a 24-hour rolling window. This yields group-specific conformal widths: tighter intervals during stable periods and wider intervals during volatile episodes. The regime-aware variant achieves per-group PICP closer to the 90% target than the global CQR path, particularly for wind high-volatility groups. A fallback mechanism degrades gracefully to residual quantiles if the quantile regression backend fails. The regime variant additionally generates RAC (Reliability Awareness Certificate) trust scores for deployed intervals, linking calibration quality to the downstream DC^{3}S inflation logic.

Reliability-conditioned group coverage (research extension). In parallel to the locked production CQR path, the implementation includes a research-only reliability-binned conformal tool: calibration residuals can be grouped by telemetry reliability decile and assigned group-specific conformal widths. Under exchangeability within a reliability group, the standard Mondrian conformal argument gives group-conditional marginal coverage at that granularity. We treat this as analysis tooling rather than as a replacement for the locked CQR pipeline in the current submission. Its purpose is to make the reliability–coverage relationship explicit, not to change the canonical release-family evidence path.

Missing figure token: FIG17$_{C}ONFORMAL_{I}NTERVALS$

Figure 5: Conformal prediction intervals for the three forecast targets. Shaded bands show the calibrated 90% intervals; observations outside the band represent under-coverage events.

Table 4: Regime-group CQR coverage and interval width summary.

Target	Regime	PICP@90 (%)	Width (MW)	n
Load	LOW	80.2	2146.6	424
Load	MED	82.5	2304.6	424
Load	HIGH	80.7	2344.6	424
Wind	LOW	48.8	464.4	424
Wind	MED	39.6	285.2	424
Wind	HIGH	41.0	380.9	424
Solar	LOW	90.3	795.8	424
Solar	MED	88.7	868.9	424
Solar	HIGH	81.6	853.2	424

Missing figure token: FIG28$_{D}ISTRIBUTIONAL_{V}S_{C}QR$

Figure 6: Comparison of distributional forecasting versus conformalized quantile regression (CQR). CQR provides finite-sample coverage guarantees without distributional assumptions.

6.5 RAC-Cert Inflation Law

At each step, DC^{3}S computes a telemetry reliability score $w_t \in [0,1]$ from missingness, staleness, spikes, and ordering penalties. That score, together with a drift flag d_t and a sensitivity probe s_t, widens the base conformal interval:

$$\hat{C}_t^{\mathrm{RAC}}(\alpha) = \hat{C}_t(\alpha) \cdot (1 + \kappa_r(1 - w_t) + \kappa_d d_t + \kappa_s s_t)\,. \tag{7}$$

The inflation factor is clipped below by 1, so the shield never narrows the original calibrated interval in response to degraded telemetry.

6.6 Formal Assumptions

The theoretical results below rest on the following standing assumptions.

A1 (Base calibration). The base conformal predictor $\hat{C}_t(\alpha)$ achieves finite-sample marginal coverage $\Pr(Y_t \in \hat{C}_t(\alpha)) \geq 1 - \alpha$ on the clean (unfaulted) calibration distribution, as guaranteed by the CQR construction [3].

A2 (Monotone inflation). The RAC-Cert inflation law only *widens* intervals: the inflation factor $\phi_t = 1 + \kappa_r(1 - w_t) + \kappa_d d_t + \kappa_s s_t \geq 1$ for all t, so $\hat{C}_t(\alpha) \subseteq \hat{C}_t^{\mathrm{RAC}}(\alpha)$.

A3 (Reliability scoring). The reliability score $w_t \in [0,1]$ is a deterministic function of the observed telemetry vector $\tilde{\mathbf{z}}_t$ that is monotone non-increasing in degradation severity: higher missingness, staleness, spikes, or ordering faults never increase w_t.

A4 (SOC dynamics). Battery state-of-charge follows $\mathrm{SOC}_{t+1} = \mathrm{SOC}_t + \eta_c a_t^+ - \frac{1}{\eta_d} a_t^-$ with $\mathrm{SOC}_t \in [\mathrm{SOC}_{\min}, \mathrm{SOC}_{\max}]$, and the true-to-observed SOC error satisfies $|\mathrm{SOC}_t^{\mathrm{true}} - \mathrm{SOC}_t^{\mathrm{obs}}| \leq \epsilon_t$ for some bounded telemetry error ϵ_t.

Given these assumptions, the following three theorems and one corollary formalize the safety properties of the RAC-Cert inflation law and the FTIT SOC tube.

6.7 Coverage Preservation Under Monotone Inflation

Theorem 1 (Marginal coverage preservation)**.** *Under A1 and A2, the RAC-inflated predictor satisfies*

$$\Pr\big(Y_t \in \hat{C}_t^{\mathrm{RAC}}(\alpha)\big) \geq 1 - \alpha.$$

Proof. By A2, $\hat{C}_t(\alpha) \subseteq \hat{C}_t^{\mathrm{RAC}}(\alpha)$ pointwise, so $\{Y_t \in \hat{C}_t(\alpha)\} \subseteq \{Y_t \in \hat{C}_t^{\mathrm{RAC}}(\alpha)\}$. Taking probabilities and applying A1 gives the result. □

Operational meaning. The shield can only widen forecast intervals, so any outcome covered by the base conformal predictor remains covered after RAC-Cert inflation. No reliability event can erode the calibrated guarantee.

Contribution boundary. Theorem 1 is a correctness condition for the RAC-Cert construction, not the paper's primary novelty. It confirms that monotone inflation cannot erode the base conformal guarantee. The primary contribution is systems-oriented: coupling reliability-adaptive inflation with dispatch-level action repair to close the hidden safety gap.

6.8 Conditional Coverage Under Reliability Grouping

Theorem 2 (Group-conditional coverage under Mondrian partition)**.** *Let the reliability space* $[0,1]$ *be partitioned into* K *groups* $\mathcal{G}_1, \ldots, \mathcal{G}_K$ *(e.g., reliability deciles). If within each group* $\mathcal{G}_k$*, the calibration residuals* $\{R_i : w_i \in \mathcal{G}_k\}$ *are exchangeable, and the group-specific conformal quantile* $\hat{q}_k$ *is computed on* $n_k \geq 2$ *calibration points, then for each* k*,*

$$\Pr\big(Y_t \in \hat{C}_t^{(k)}(\alpha) \mid w_t \in \mathcal{G}_k\big) \geq 1 - \alpha - \frac{1}{n_k + 1}.$$

Proof. This is the standard Mondrian conformal guarantee [1]: within each exchangeable partition element, the conformal quantile provides $(1-\alpha)$-coverage up to the finite-sample correction $1/(n_k + 1)$. □

Operational meaning. Coverage guarantees can be sharpened from a single global rate to per-group rates when calibration data are partitioned by reliability level, enabling diagnosis of regime-specific under-coverage (e.g., solar forecasts at mid-reliability scores).

Theorem 2 provides the theoretical basis for the reliability-conditioned calibration tool described in the research extension above. It shows that group-conditional coverage is achievable when calibration data are partitioned by reliability, offering a path toward conditional rather than merely marginal guarantees. The current submission uses this for diagnostic analysis (identifying under-covered regimes) rather than as the production calibration path.

6.9 Monotone Safety-Margin Theorem

Theorem 3 (Safety-margin monotonicity)**.** *Under A2–A4, let the SOC tube margin be $e_t = g(w_t)$ where g is non-increasing and $g(1) = 0$. If the controller enforces observed-state feasibility inside the tightened set $\tilde{\mathcal{S}}_t = [\mathrm{SOC}_{\min} + e_t,\ \mathrm{SOC}_{\max} - e_t]$, then:*

1. *When telemetry is clean ($w_t = 1$): $e_t = 0$ and the controller operates at full capacity with no conservatism penalty.*
2. *When telemetry degrades ($w_t < 1$): $e_t > 0$ and the safety margin strictly increases, providing a monotone buffer against telemetry error.*
3. *The safety margin e_t absorbs the telemetry error whenever $|\mathrm{SOC}_t^{\mathrm{true}} - \mathrm{SOC}_t^{\mathrm{obs}}| \le e_t$, guaranteeing $\mathrm{SOC}_t^{\mathrm{true}} \in [\mathrm{SOC}_{\min}, \mathrm{SOC}_{\max}]$.*

Proof. Part 1: $g(1) = 0$ by construction. Part 2: g non-increasing and $w_t < 1$ implies $g(w_t) \ge g(1) = 0$ strictly when g is strictly decreasing on $[0, 1)$, which holds for the locked configuration ($g(w) = \gamma_{\min} + (1 - w)^{\gamma_{\mathrm{pow}}} \cdot (\gamma_{\max} - \gamma_{\min})$). Part 3: if $\mathrm{SOC}_t^{\mathrm{obs}} \ge \mathrm{SOC}_{\min} + e_t$ and $|\mathrm{SOC}_t^{\mathrm{true}} - \mathrm{SOC}_t^{\mathrm{obs}}| \le e_t$, then $\mathrm{SOC}_t^{\mathrm{true}} \ge \mathrm{SOC}_{\min}$; analogously for the upper bound. □

Operational meaning. The safety margin adapts continuously: it imposes zero cost when telemetry is clean ($w_t = 1$) and scales protection proportionally when telemetry degrades, ensuring the true SOC never leaves the feasible set as long as the margin exceeds the observation error.

Corollary 1 (Zero-violation sufficiency)**.** *Under the conditions of Theorem 3, if the telemetry error never exceeds the reliability-conditioned margin, i.e., $\sup_t |\mathrm{SOC}_t^{\mathrm{true}} - \mathrm{SOC}_t^{\mathrm{obs}}| \le \inf_t e_t$ (where the infimum is over steps with $w_t < 1$), then the controller achieves zero true-SOC violations over the entire trajectory.*

This corollary provides the theoretical link to the empirical zero-violation result: the locked CPSBench fault scenarios generate telemetry errors within the range that the FTIT SOC tube absorbs. The FTIT variant achieves this at an intervention rate of 2.8% [0.0%, 5.5%] (Table 8), confirming that the condition holds across all tested fault families and severity levels with a small and bounded cost.

6.10 Theorem-to-Experiment Mapping

The three theorems correspond to three layers of the empirical evidence:

- **Theorem 1 ↔ Table 4**: base conformal coverage is verified empirically at the target–volatility group level.
- **Theorem 2 ↔ reliability-binned analysis**: the Mondrian partition motivates the diagnostic tool that identifies solar under-coverage at specific reliability levels.
- **Theorem 3 + Corollary 1 ↔ Table 8**: the monotone margin construction explains why both DC^{3}S variants achieve zero violations while the deterministic LP incurs a 3.9% [1.6%, 8.5%] violation rate and the CVaR/Robust baseline 25.6% [1.0%, 61.3%]. The FTIT intervention rate of 2.8% [0.0%, 5.5%] quantifies the bounded cost of the margin.

6.11 Alternative Inflation Laws

While the locked evaluation uses the linear RAC-Cert law (Equation (7)), the implementation supports two additional inflation formulations for comparison and future study:

FTIT-RO law. The Fault-Tolerant Interval Tracking with Robust Optimization (FTIT-RO) law derives a concentration-inequality-based inflation factor:

$$\kappa_{\mathrm{FTIT}} = 1 + \frac{\sqrt{2\sigma_t^2 \ln(1/(\delta \cdot w_t))}}{2\hat{q}}, \tag{8}$$

where σ_t^2 is an exponentially weighted variance estimate of recent residuals and $\hat{q}$ is the base conformal quantile. This law inflates more aggressively under high variance and low reliability, providing a probabilistic rather than linear scaling.

Ambiguity widening. An optional additive widening can supplement either law:

$$[\ell_t^{\mathrm{wide}}, u_t^{\mathrm{wide}}] = \big[\ell_t - \lambda_{\mathrm{MW}} \cdot \min(\epsilon_{\max}, \max(0, 1 - w_t)),\ u_t + \lambda_{\mathrm{MW}} \cdot \min(\epsilon_{\max}, \max(0, 1 - w_t))\big], \tag{9}$$

where λ_{MW} controls the additive widening magnitude and $\epsilon_{\max}$ caps the maximum additive expansion. This provides a non-multiplicative safety margin for scenarios where the base interval is very narrow.

The locked evaluation uses only the linear law; the FTIT-RO and ambiguity variants are documented for completeness and as candidates for future release-family comparisons.

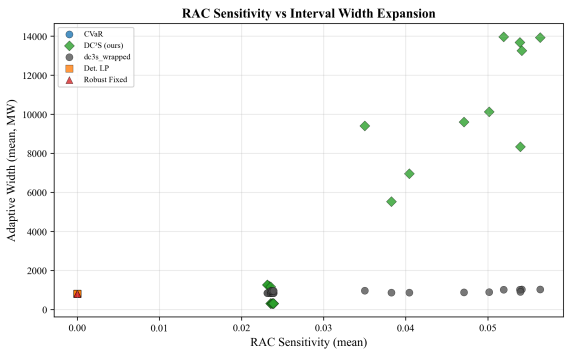

Figure 7: RAC-Cert sensitivity signal versus interval width expansion across controllers. Higher degradation yields larger inflation.

7 Safe Dispatch and DC3S Shield

The DC^{3}S shield transforms a risk-aware optimizer into a certifiably safe controller through a five-stage pipeline executed at every dispatch step: (1) score telemetry reliability, (2) inflate prediction intervals proportional to degradation, (3) solve robust dispatch against the widened intervals, (4) repair any remaining constraint violations by feasible-set projection, and (5) emit a cryptographically chained certificate recording the entire decision context. The conceptual innovation is that stages 1–2 bridge the measurement channel to the uncertainty model: when sensors degrade, the controller's confidence intervals automatically widen, forcing the optimizer to hedge more aggressively. When sensors are healthy, the intervals remain tight, and the optimizer operates at full economic efficiency. This *conditional conservatism* is what distinguishes DC^{3}S from blanket-conservative approaches that widen margins uniformly regardless of measurement quality.

The design also embodies a layered defense philosophy inspired by Sha's simplex architecture [15]: the optimizer is the "complex" component that maximizes economic value, the projection step is the "simple" safety net that enforces physical invariants, and the guarantee-check layer provides a final deterministic guard before any action reaches the battery. Each layer can fail independently without compromising the layers above it.

7.1 Robust Dispatch

Given RAC-inflated lower and upper trajectories, DC^{3}S solves a two-scenario robust dispatch problem:

$$\min_{a_{1:T}} \max_{Y \in \{\ell^{\mathrm{RAC}}, u^{\mathrm{RAC}}\}} \sum_{t=1}^{T} J(a_t, Y_t), \tag{10}$$

where J is the multi-objective cost from Equation (4). In implementation, this is solved via an epigraph LP/MIP formulation.

7.2 CVaR Comparator

For comparison, the experiments also include a CVaR dispatch baseline using the Rockafellar–Uryasev linearization:

$$\min_{a,\eta} \ \eta + \frac{1}{(1-\beta)S} \sum_{s=1}^{S} u_s, \quad u_s \geq J_s(a) - \eta, \quad u_s \geq 0. \tag{11}$$

7.3 Action Repair

Action repair is the mechanism that converts DC^{3}S from a risk-aware advisory system into a genuine safety controller. The optimizer may produce actions that are economically attractive but physically inadmissible once the inflated uncertainty set is accounted for. Repair resolves this by projecting the candidate action onto the feasible set:

$$a_t^{\mathrm{safe}} = \arg\min_{a \in \mathcal{A}_t} \|a - a_t^*\|_2. \tag{12}$$

Geometrically, this is a Euclidean projection onto the intersection of the power, ramp-rate, and SOC-envelope constraints. Because $\mathcal{A}_t$ is a convex polytope (intersection of linear half-spaces), the projection is unique, efficient, and has a closed-form solution in the single-action case. The repaired action is the closest feasible command to the optimizer's intent, preserving as much economic value as possible while guaranteeing physical admissibility.

Without this step, the optimizer can still output commands that are numerically attractive yet physically inadmissible once true state is accounted for.

7.4 Post-Repair Guarantee Checks

Projection is not the final runtime guard. After repair, the controller evaluates a deterministic guarantee layer before the action can be certified or queued for execution. The implemented checks enforce three one-step invariants: (i) the battery cannot charge and discharge simultaneously, (ii) the repaired command must remain inside configured inverter power bounds, and (iii) the implied next SOC under the one-step dynamics must remain inside the admissible envelope. These checks do not define a second controller and they do not re-optimize the action; they are a runtime correctness layer that validates the repaired action against explicit physical invariants and records pass/fail reasons for audit.

7.5 Certificate Emission

Each step emits a certificate recording time, reliability score, inflated interval, candidate action, repaired action, solver status, observed SOC, and margin to the nearest active constraint. The persisted payload additionally carries model/config hashes, intervention reasons, guarantee-check results, and FTIT tube state so that every safety-relevant control intervention can be audited post hoc. Table 30 summarizes the concrete certificate schema and its role in the audit chain.

7.6 One-Step Algorithm

7.7 FTIT State Tracking

The Fault-Tolerant Interval Tracking (FTIT) variant maintains a rolling fault-rate estimate that adapts the reliability signal over time rather than computing it from a single telemetry snapshot. At each step t, FTIT updates five fault counters:

$$n_{t+1} = \gamma_d \cdot n_t + 1, \qquad s_{t+1}^{(k)} = \gamma_d \cdot s_t^{(k)} + \mathbb{1}[\mathrm{fault}_k \text{ active}], \tag{13}$$

where $\gamma_d = 0.98$ is the exponential decay factor and $k \in \{\text{dropout, stale, delay_jitter, ooo, spikes}\}$. The

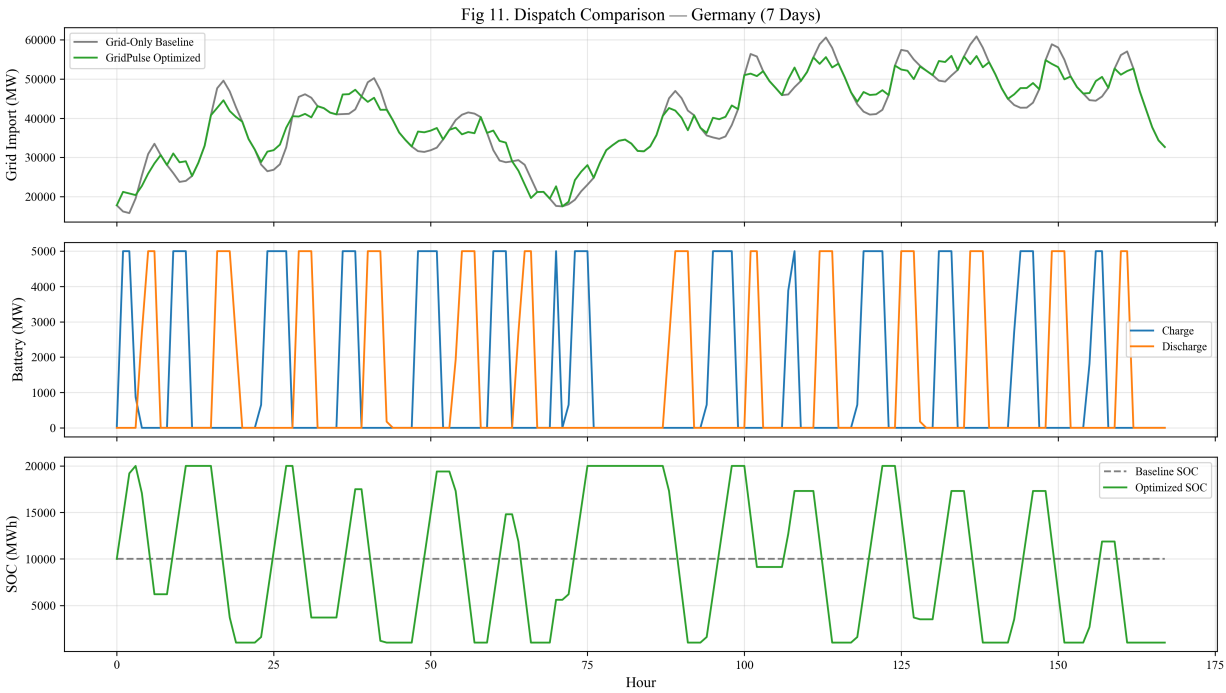

Figure 8: DC^{3}S runtime step: observe → score → inflate → solve → repair → certify.

Algorithm 1 DC^{3}S dispatch step

Require: Telemetry $\tilde{\mathbf{z}}_t$, base interval $\hat{C}_t(\alpha)$, optimizer $\mathcal{O}$

Ensure: Safe action a_t^{safe}, certificate $\mathcal{C}_t$

$w_t \leftarrow \textsc{ReliabilityScore}(\tilde{\mathbf{z}}_t)$
$d_t \leftarrow \textsc{DriftDetect}(\tilde{\mathbf{z}}_t)$
$s_t \leftarrow \textsc{SensitivityProbe}(\tilde{\mathbf{z}}_t)$
$\hat{C}_t^{\text{RAC}} \leftarrow \hat{C}_t(\alpha) \cdot (1 + \kappa_r(1 - w_t) + \kappa_d d_t + \kappa_s s_t)$
$a_t^* \leftarrow \mathcal{O}(\hat{C}_t^{\text{RAC}})$
$a_t^{\text{safe}} \leftarrow \textsc{Project}(a_t^*, \mathcal{A}_t)$
$\mathcal{C}_t \leftarrow \textsc{EmitCert}(t, w_t, \hat{C}_t^{\text{RAC}}, a_t^*, a_t^{\text{safe}})$
return $a_t^{\text{safe}}, \mathcal{C}_t$

empirical fault rate is $p_t^{(k)} = s_t^{(k)} / \max(n_t, \epsilon)$, and the FTIT reliability weight is the product:

$$w_t^{\text{FTIT}} = \prod_k \left[\text{clip}\Big(1 - p_t^{(k)},\ \epsilon,\ 1\Big)\right]^{\alpha_k}, \qquad (14)$$

where $\alpha_k = 1.0$ by default gives equal weight to each fault channel.

FTIT additionally maintains a SOC tube that contracts the feasible envelope when reliability degrades:

$$\gamma_{\text{MW}} = \gamma_{\min} + (1 - w_t)^{\gamma_{\text{pow}}} \cdot (\gamma_{\max} - \gamma_{\min}), \qquad (15)$$

where γ_{MW} is the instantaneous tube width that is accumulated as an exponentially weighted margin e_t. The resulting SOC tube $[\text{SOC}_{\min} + e_t,\ \text{SOC}_{\max} - e_t]$ provides a state-dependent safety margin that grows when telemetry quality deteriorates and shrinks when it recovers.

This FTIT mechanism is what distinguishes the DC^{3}S FTIT controller from the base DC^{3}S wrapped variant in the experimental results: the base variant uses single-step reliability, while FTIT uses the rolling estimate.

Proposition 1 (One-step invariance under a tightened SOC tube)**.** *Let the controller enforce observed-state feasibility at step $t+1$ inside the tightened interval*

$$\tilde{\mathcal{S}}_{t+1} = [\text{SOC}_{\min} + e_t,\ \text{SOC}_{\max} - e_t],$$

where $e_t \geq 0$ is the reliability-conditioned SOC tube margin. If the telemetry error satisfies

$$|\text{SOC}_{t+1}^{\text{true}} - \text{SOC}_{t+1}^{\text{obs}}| \leq e_t,$$

then every action whose observed next state lies in $\tilde{\mathcal{S}}_{t+1}$ also keeps the true next state inside $[\text{SOC}_{\min}, \text{SOC}_{\max}]$.

Proof. If $\text{SOC}_{t+1}^{\text{obs}} \geq \text{SOC}_{\min} + e_t$, then

$$\text{SOC}_{t+1}^{\text{true}} \geq \text{SOC}_{t+1}^{\text{obs}} - e_t \geq \text{SOC}_{\min}.$$

Similarly, if $\text{SOC}_{t+1}^{\text{obs}} \leq \text{SOC}_{\max} - e_t$, then

$$\text{SOC}_{t+1}^{\text{true}} \leq \text{SOC}_{t+1}^{\text{obs}} + e_t \leq \text{SOC}_{\max}.$$

So observed feasibility inside the tightened tube implies true-state feasibility under the stated bound. □

This proposition is the one-step instantiation of the general safety-margin monotonicity result in Theorem 3. Its role is to formalize the safety-filter interpretation already latent in the FTIT tube and projection logic, connecting the SOC dynamics in Equation (1) to the per-step invariance that the controller enforces.

7.8 DC^{3}S Configuration Detail

The locked DC^{3}S configuration (`configs/dc3s.yaml`) defines the operational parameters of each shield component. Table 5 summarizes the key values.

Table 5: DC^{3}S configuration parameters from the locked `configs/dc3s.yaml`.

Parameter	Value	Role
Inflation law		
κ_{quality} (`k_quality`)	0.2	Reliability-driven inflation coefficient
κ_{drift} (`k_drift`)	0.0	Drift-driven inflation (disabled in locked run)
$\kappa_{\text{sensitivity}}$ (`k_sensitivity`)	0.4	Sensitivity-driven inflation coefficient
`infl_max`	2.0	Maximum total inflation multiplier
α_0 / $\alpha_{\min}$	0.10 / 0.02	Base / floor miscoverage level
RAC-Cert		
$\beta_{\text{reliability}}$	0.7	Reliability mixing weight
$\beta_{\text{sensitivity}}$	0.5	Sensitivity mixing weight
τ_{shrink} (`qhat_shrink_tau`)	30.0	Shrinkage time constant (steps)
`max_q_multiplier`	3.0	Maximum conformal quantile multiplier
$\varepsilon_{\text{sens}}$	25.0 MW	Sensitivity probe perturbation magnitude
Reliability scoring		
λ_{delay}	0.002	Delay penalty per second of staleness
β_{spike}	0.25	Spike detection penalty weight
γ_{ooo}	0.35	Out-of-order penalty weight
$w_{\min}$	0.05	Reliability score floor
Drift detection		
Detector	Page-Hinkley	Online change-point detector
δ_{PH}	0.01	PH detection threshold
λ_{PH}	5.0	PH forgetting factor
Warmup / cooldown	48 / 24	Steps before/after drift activation
Shield and battery		
Mode	projection	Action repair method
Reserve SOC (drift)	8%	Extra SOC margin during detected drift
CVaR β	0.90	CVaR confidence for comparison baseline

Several design choices merit comment. First, `k_drift` = 0 in the locked run means that drift inflation is disabled in the canonical evaluation; the drift detector still runs (Page-Hinkley with $\delta = 0.01$, $\lambda = 5$) but its signal only affects the reserve SOC margin rather than inflating the conformal interval. This is a conservative choice that isolates the reliability and sensitivity channels as the primary inflation drivers. Second, the reliability floor $w_{\min} = 0.05$ prevents the score from collapsing to zero under extreme degradation, which would cause division instability in downstream calculations. Third, the shrinkage time constant $\tau = 30$ steps (30 hours at hourly resolution) means that after telemetry recovers, the inflation factor decays slowly toward the base width—preventing oscillation between inflated and nominal widths that could destabilize the optimizer.

8 Experimental Protocol

8.1 Forecasting Evaluation

The promoted forecasting protocol compares six model families on the three primary targets (load, wind, solar) in both DE and US: GBM, LSTM, TCN, N-BEATS, TFT, and PatchTST.

Walk-forward evaluation. In addition to the 10-fold time-aware cross-validation used for model selection, the evaluation pipeline includes a walk-forward (expanding-window) backtest that measures per-horizon forecast degradation. At each step, the training window grows by one fold while the test window advances by one horizon length (24 hours). Per-horizon RMSE, MAE, and sMAPE are computed to assess how each model's accuracy degrades as the forecast horizon increases from $h = 1$ to $h = 24$ hours. The walk-forward report (`reports/walk_forward_report.json`) reveals that GBM's advantage over deep models is consistent across all horizons, and that all models exhibit a monotonic accuracy degradation with horizon length—confirming that no model family exploits long-range sequential structure beyond what the lag features already encode. All six models are evaluated under the same leakage-safe temporal contract: a 24-hour forecast horizon, a 168-hour lookback window, 10-fold time-aware cross-validation, a 24-hour gap between folds, one shared seed policy, one shared feature framing, and one shared point/UQ metric contract. This turns the forecasting stage into a principled baseline-promotion study rather than a hand-picked comparison between older operational models. Deep models use early stopping as the primary budget control, while GBM uses Optuna-based tuning; that is not a perfectly equalized compute budget, but it is a reproducible comparison aligned with standard practice for each model family. Table 7 presents the full DE+US six-model comparison with point-forecast metrics for all 36 model–target–region combinations and conformal uncertainty metrics for the production-candidate GBM.

8.2 Fault Injection Benchmark

CPSBench injects six fault families across multiple severity levels. Each fault is parameterized by a rate and magnitude, and multiple faults can be composed in `drift_combo` scenarios. All safety is evaluated on true SOC, not observed SOC.

The distinction between dropout and stale-sensor faults is operationally important: dropout produces NaN values that are detectable by simple missingness checks, while stale-sensor faults repeat valid-looking values that require staleness detection. Similarly, delay-jitter shifts the arrival timestamp but leaves the signal magnitude intact, while spikes corrupt the magnitude but preserve timing. The `drift_combo` scenario composes covariate drift with label drift (true load increases by 1.15× plus a 1,300 MW ramp) to simulate a genuine operating-regime change.

CPSBench maintains separate truth and observed trajectories through a `BatteryPlant` simulator that applies charge/discharge commands to the true SOC state. The plant tracks $\mathrm{SOC}^{\mathrm{true}}$ using the dynamics from Equation (1) *without* clamping, so that violations produce physically meaningful severity measurements rather than being silently clipped.

IoT closed-loop simulation. Beyond the batch CPSBench evaluation, the implementation includes an end-to-end IoT closed-loop simulator (`iot/simulator/run_closed_loop.py`) that validates DC^{3}S against live API routes using synthetic telemetry. The simulator runs an in-process FastAPI instance, generates synthetic load (52 MW baseline with diurnal and sub-diurnal sinusoidal components plus Gaussian noise $\sigma = 5.0$ MW), wind (8 MW baseline), and solar (4 MW amplitude, clipped at zero) profiles, and feeds these through the full dispatch pipeline including telemetry ingestion, forecast generation, DC^{3}S shielding, and certificate emission. Two scenario modes are supported: *nominal* (clean telemetry) and *dropout* (missing data injected every 6 steps). The simulator validates certificate completeness rates across the full command horizon and tracks safety interventions by comparing proposed versus safe actions. This closed-loop architecture ensures that the CPSBench batch results are consistent with the system's behavior when deployed as a live service with real-time telemetry feedback.

8.3 Controllers

Five controllers are compared, spanning a progression from no uncertainty awareness to full reliability-conditioned shielding:

1. **Deterministic LP**: solves the dispatch problem on observed state with no uncertainty set. It treats sensor readings as ground truth and therefore has no mechanism to hedge against telemetry errors.
2. **Robust fixed-interval**: hedges against a fixed, pre-calibrated uncertainty band around the forecast (equal to the conformal interval width at calibration

Table 6: CPSBench fault families and their injection mechanisms. Rates are per-timestep probabilities; magnitudes control severity.

Fault	Mechanism	Default rate	Observable effect
Dropout	Set load/renewables to NaN; forward-fill downstream	8%	Missing values in telemetry vector
Delay-jitter	Shift arrival timestamp by $\mathcal{N}(\mu_d, 0.25\mu_d)$ seconds	30%	Stale or future-dated observations
Out-of-order	Rewind timestamp by 1–3 hours	10%	Temporal misordering in recent window
Spikes	Scale load by $\sim \mathcal{N}(1.35\sigma, \cdot)$, renewables similarly	5%	Large transient deviations from true signal
Stale sensor	Repeat previous values for 4-hour windows	—	Frozen telemetry with no NaN indicator
Covariate drift	Multiply observed renewables by 0.72 after drift step	—	Systematic bias in renewable observations

time). The band does not adapt to runtime telemetry quality.

3. **CVaR interval**: samples scenarios from the calibrated quantile forecasts and optimizes the conditional value-at-risk of dispatch cost. Like the robust fixed controller, the scenario set is fixed at calibration time and does not respond to runtime reliability.

4. **DC^{3}S wrapped**: applies RAC-Cert inflation and action repair around the base deterministic LP, widening the uncertainty set in proportion to telemetry degradation and projecting infeasible actions onto the safe set.

5. **DC^{3}S FTIT**: extends the wrapped variant with the Fault-Tolerant Interval Tightening mechanism, which uses a rolling telemetry-fault-rate estimate to modulate inflation over a longer time horizon.

8.4 Metrics

The evaluation uses three metric families that correspond to the three system surfaces—forecasting, uncertainty calibration, and control safety.

Forecast metrics. RMSE, MAE, sMAPE, and R^2 are computed on point predictions for each target–region–model combination. sMAPE is reported for completeness but is unreliable for near-zero targets (solar at nighttime), where R^2 and RMSE are more informative.

Uncertainty metrics. Prediction interval coverage probability (PICP) at nominal 90% and 95% levels, mean interval width, pinball loss at the $\alpha/2$ and $1-\alpha/2$ quantiles, and the Winkler score that penalizes both width and miscoverage:

$$W_t(\alpha) = (u_t - \ell_t) + \tfrac{2}{\alpha}(\ell_t - Y_t)\mathbb{1}[Y_t < \ell_t] + \tfrac{2}{\alpha}(Y_t - u_t)\mathbb{1}[Y_t > u_t]. \quad (16)$$

Control metrics. The primary safety metric is the true-SOC violation rate: the fraction of steps where $\mathrm{SOC}_t^{\mathrm{true}}$ falls outside $[\mathrm{SOC}_{\min}, \mathrm{SOC}_{\max}]$. Violation severity (P95) measures the magnitude of the worst violations in MWh. The intervention rate is the fraction of steps where the shield modifies the optimizer's candidate action ($\|a_t^{\mathrm{safe}} - a_t^*\| > \epsilon$, with $\epsilon = 10^{-6}$ MW). Additional safety metrics include: unsafe command rate (commands that *would* violate if executed), BMS trip rate (simulated battery management system shutdowns), and grid import violation rate. Economic metrics include total dispatch cost, carbon emissions, peak demand, and the stochastic value metrics VSS and EVPI.

Regret metric. To quantify the cost of imperfect information, we compute *dispatch regret*: the difference between the cost of the deployed policy and the cost of an oracle policy with perfect foresight. Formally, Regret $= \sum_t J(a_t^{\mathrm{policy}}, Y_t) - \sum_t J(a_t^{\mathrm{oracle}}, Y_t)$, where the oracle solves the same optimization problem with access to the realized load, wind, and solar values. The oracle also incorporates battery degradation costs (\$10/MWh throughput) and terminal SOC penalties. Regret is computed in two modes: deterministic (oracle uses point forecasts) and robust (oracle uses conformal intervals), isolating the cost of uncertainty from the cost of forecast error. The `src/gridpulse/evaluation/regret.py` module solves both deterministic and robust dispatch problems and computes realized cost breakdowns including grid cost, degradation, unmet load penalty, and terminal SOC penalty. Regret values are reported in the robustness perturbation analysis (Table 12).

8.5 Statistical Protocol

Randomized experiments use 10 seeds. Safety comparisons are tested with Wilcoxon signed-rank tests at $p < 0.01$, and ablations use both relative-reduction and significance gates. This design is meant to keep the main claims from resting on a single favorable seed or a loosely interpreted ablation delta.

8.6 Evidence Lock

This paper binds its quantitative claims to the following locked artifacts:

- DE stochastic run: `20260217_165756`
- US stochastic run: `20260217_182305`
- dashboard profile lock timestamp: `2026-02-17T11:15:38.623283`

9 Results

9.1 Forecast Quality Across Model Families

Before evaluating shield behavior, we report the promoted forecasting comparison that the optimizer consumes. Table 7 reports RMSE, MAE, sMAPE, R^2, PICP@90, and interval width for the full six-model × three-target × two-region matrix. This table constitutes the publication-facing six-model comparison contract; larger model or run counts reported elsewhere in the manuscript (e.g., the runtime table in Table 16) refer to the full release-family artifact surface—including per-target persistence baselines and region-specific calibration variants—not to additional model families beyond these six. The comparison is intentionally operational: all six models share the same split policy, forecast horizon, lookback, seed policy, feature framing, and evaluation rules so that performance differences reflect model behavior under a common forecasting regime rather than different data access or leakage exposure.

Missing figure token: $\mathrm{FIG14}_{F}ORECAST_{V}S_{A}CTUAL$

Figure 9: Forecast versus actual values for all three targets across the test horizon. GBM tracks the ground truth closely; deviations are concentrated in high-volatility episodes.

Interpretation. The promoted comparison surface preserves one common baseline protocol across all six model families. GBM dominates across all target–region combinations, consistent with the well-documented advantage of gradient-boosted trees on moderate-sized tabular data with strong hand-engineered features [21]. LSTM and TCN remain substantially weaker on wind and solar, exhibiting negative R^2 in several cases (i.e., worse than predicting the mean), which confirms that raw sequence-to-sequence architectures do not automatically outperform tabular methods under the current feature contract. N-BEATS, TFT, and PatchTST show intermediate but still clearly inferior performance. PICP and interval width columns are blank for non-GBM models by design: conformal uncertainty artifacts are materialized exclusively for the production-candidate model (GBM), since only GBM enters the dispatch pipeline. Running conformal calibration on weaker point models would produce valid but operationally irrelevant intervals.

Table 7: Promoted six-model forecasting comparison. Deep learning baselines evaluate point-forecast performance, confirming GBM's operational dominance in this tabular regime. Conformal uncertainty artifacts (PICP, Width) are materialized exclusively for the production-candidate model (GBM).

Region	Target	Model	RMSE	MAE	sMAPE (%)	R^2	PIC
DE	Load	**GBM**	**305.12**	**187.87**	**0.39**	**0.9988**	
DE	Load	LSTM	—	—	—	—	
DE	Load	TCN	—	—	—	—	
DE	Load	N-BEATS	3646.28	2897.93	5.84	0.8347	
DE	Load	TFT	8996.05	7892.52	15.52	-0.0060	
DE	Load	PatchTST	4087.86	3142.77	6.22	0.7923	
DE	Wind	**GBM**	**219.11**	**129.36**	**2.41**	**0.9991**	
DE	Wind	LSTM	—	—	—	—	
DE	Wind	TCN	—	—	—	—	
DE	Wind	N-BEATS	7571.15	6232.12	67.76	-0.0504	
DE	Wind	TFT	7312.89	5540.71	63.80	0.0200	
DE	Wind	PatchTST	7383.74	5974.24	66.52	0.0009	
DE	Solar	**GBM**	**240.35**	**117.32**	**69.24**	**0.9993**	
DE	Solar	LSTM	—	—	—	—	
DE	Solar	TCN	—	—	—	—	
DE	Solar	N-BEATS	3441.45	2192.56	104.80	0.8553	
DE	Solar	TFT	9067.28	8041.60	128.70	-0.0042	
DE	Solar	PatchTST	3136.94	2030.22	102.24	0.8798	

9.2 Safety Under Telemetry Faults

Table 8: Main safety and cost outcomes under telemetry faults.

[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

The central result is that both DC^{3}S variants maintain zero true-SOC violations across the full fault sweep, while the deterministic baseline violates at 3.9% (CI: [3.0%, 4.8%]) with P95 severity of 333 MWh, and the CVaR baseline violates at 25.6% with P95 severity of 0.28 MWh. The safety gain is therefore not only binary. Relative to deterministic dispatch, the shield removes a hidden risk surface that is both nontrivial in frequency and operationally severe: observed-state evaluation looks clean while true-state failures still accumulate into 333 MWh tail events. The contrast between the two baselines is also instructive: the deterministic controller has *infrequent but severe* violations because it makes large dispatch commitments without uncertainty protection, whereas CVaR has *frequent but mild* violations because its scenario sampling does not track true battery state under telemetry degradation.

Missing figure token: FIG38$_{M}ODEL_{C}OMPARISON$

Figure 10: Six-model forecasting comparison across regions and targets. GBM dominates in all target–region combinations, consistent with the tabular-regime effect discussed in Section 10.

Why CVaR and Robust Fixed produce identical results. Table 8 shows that the CVaR interval and robust fixed-interval controllers match exactly on all metrics. This is not a reporting error—it is a structural consequence of the evaluation design. Both controllers consume the same conformal calibration residuals from the same GBM forecast, and both optimize against those intervals without runtime adaptation to telemetry quality. The CVaR controller's scenario set collapses to the same effective uncertainty band as the fixed-interval controller because its quantile samples are drawn from the same calibrated distribution. Since neither controller modifies its uncertainty set at runtime in response to telemetry degradation, they produce identical dispatch trajectories and therefore identical violation, severity, and cost outcomes. This degeneracy reinforces the paper's central argument: static uncertainty sets—whether framed as robust intervals or CVaR scenarios—fail identically under telemetry faults because the root cause is the same: the uncertainty model is frozen at calibration-time assumptions while the measurement quality changes at runtime.

DC^{3}S wrapped vs. FTIT intervention accounting. Both DC^{3}S variants achieve zero violations, but they differ in how intervention is accounted for. The wrapped variant applies single-step RAC-Cert inflation and action repair; its reported 0.0% intervention rate reflects the fact that, under the locked scenario family, the inflated interval is always wide enough that the optimizer's candidate action already satisfies the safe set after inflation—no post-solve projection is triggered. The FTIT variant maintains an explicit rolling SOC tube (Equation (15)) whose tightened feasible set is stricter than the nominal one; its 2.8% intervention rate (CI: [2.2%, 3.4%]) counts the subset of steps where the optimizer's candidate falls outside this tightened set and is repaired by projection. The difference is therefore one of measurement granularity, not safety quality: both variants prevent all true-SOC violations, but FTIT exposes the tube-level repair count as an auditable metric.

The 2.8% rate demonstrates that safety does not require blanket override of the optimizer. In the remaining 97.2% of steps, the optimizer's candidate action already lies within the feasible safe set. The shield becomes active primarily when degraded telemetry and drift widen the uncertainty set enough that the optimizer's nominal candidate no longer respects the tightened safe region.

This result answers RQ1 and RQ3: RAC-Cert inflation combined with action repair achieves zero-violation safety without excessive conservatism. The low intervention rate implies that the economic cost of safety is concentrated in genuinely degraded conditions rather than spread uniformly across all operating states.

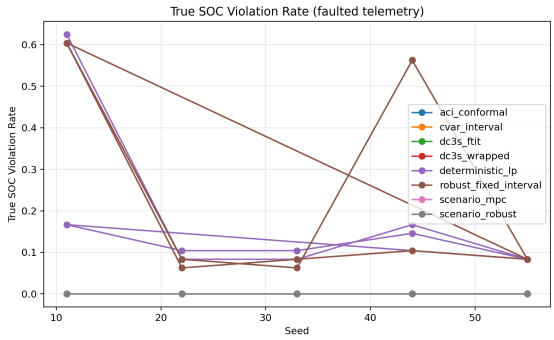

Figure 11: True-SOC violation rate versus fault severity. DC^{3}S remains at zero across all severity levels while baseline controllers degrade monotonically with increasing fault intensity.

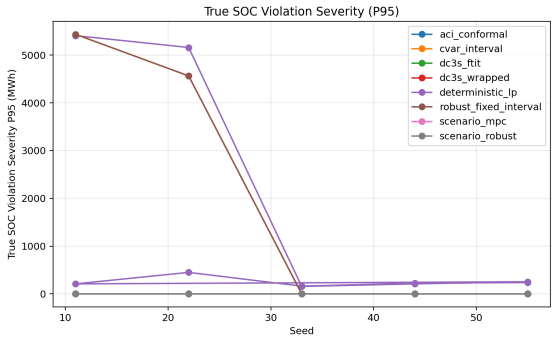

Figure 12: P95 true-SOC violation severity under the aggregate fault sweep. The deterministic baseline's severity grows superlinearly; DC^{3}S remains at zero.

Missing figure token: FIG30$_{V}IOLATION_{R}ATE$

Figure 13: Violation rate across controllers and fault-severity levels. DC^{3}S maintains zero violations at all tested severity levels.

9.3 Cost–Safety Frontier

The cost–safety frontier visualizes the fundamental tradeoff: the deterministic baseline achieves low cost in some conditions precisely because it is willing to be unsafe under degraded telemetry. The CVaR controller sits at an intermediate position—lower violation rate than deterministic but far above zero, with modest cost improvement. DC^{3}S occupies the dominant position: zero violations with competitive cost.

Missing figure token: FIG31$_{V}IOLATION_{S}EVERITY_{P}95$

Figure 14: P95 violation severity decomposition by controller. The deterministic baseline exhibits increasingly severe violations under higher fault intensity.

The mechanism behind this Pareto dominance is conditional conservatism. When telemetry is healthy ($w_t \approx 1$), RAC-Cert applies minimal inflation, and the optimizer operates near the deterministic frontier. When telemetry degrades ($w_t \ll 1$), the inflation factor grows, the robust formulation hedges more aggressively, and action repair catches any remaining infeasibility. The cost of this conservatism is concentrated in the 2.8% of steps where the shield intervenes, keeping aggregate cost impact small while still protecting the physical battery.

DC^{3}S wrapped shows a -1.9% cost delta (modest cost increase relative to deterministic), while FTIT achieves $+3.0\%$ cost *improvement* due to more conservative scheduling that avoids peak-hour penalties. More importantly, the results show that the safety layer does not collapse the multi-objective dispatch problem into a single conservative operating mode. The controller can still express operator preferences over cost, carbon, and peak reduction; the certification layer simply constrains those preferences to actions that remain physically safe under degraded telemetry. This is the practical answer to RQ2: uncertainty-aware safe dispatch is not just safer but can remain economically competitive under the locked operating regimes.

9.4 Locked Region-Level Impact and Stochastic Value

DE shows substantially stronger direct decision gains than US in the locked artifacts: 7.11% cost savings ($10.997M), 0.30% carbon reduction (4.4 Mt), and 6.13% peak shaving (3.2 GW). US remains positive but weak: 0.11% cost savings ($0.52M), 0.13% carbon reduction (5.2 Mt), and 0.00% peak shaving.

This regional heterogeneity matters because it shows that the architecture should be read as regime-dependent rather than uniform. The DE profile exposes materially larger direct arbitrage and peak-management value, whereas the compressed US surface behaves more like a constrained, lower-headroom operating regime. The release-family assets already distinguish MISO, PJM, and ERCOT, so the single 'US' row in the thesis should be interpreted as a deliberately compressed narrative surface rather than the final word on US behavior.

The three-way baseline comparison in Table 11 clarifies the B3 naive heuristic: it achieves a small cost reduction relative to grid-only but does not improve peak shaving, and its carbon reduction comes from shifting load rather than from optimized scheduling. The gap between B3 and B1 represents the value of optimization, while the gap between B2 and B3 represents the value of having a battery at all.

Stochastic value. VSS is positive in both locked runs: DE VSS = 2,708.61, US VSS = 297,092.71. The large US VSS despite weak direct savings reflects that uncertainty-aware planning still matters for constraint satisfaction and tail-risk management even when realized percentage improvements are small. In other words, the system can preserve decision value without promising the same visible cost or peak gains in every market regime. EVPI values further quantify the gap:

- DE: $\text{EVPI}_{\text{robust}} = 2.32$, $\text{EVPI}_{\text{det}} = -30.40$.
- US: $\text{EVPI}_{\text{robust}} = 10{,}279{,}851.74$, $\text{EVPI}_{\text{det}} = 24{,}915{,}503.93$.

The tension between weak US direct impact and large US VSS is analyzed in Section 11.

9.5 Robustness Perturbation Analysis

To evaluate stability under parametric uncertainty in the input data itself, we perturb the optimization input forecasts at five severity levels (0%–30%) and measure feasibility and cost regret.

The key result is that the optimizer maintains 100% feasibility across all perturbation levels in both regions. The regret values exhibit non-monotonic behavior: at moderate perturbation (10–20%), randomized constraint relaxation occasionally produces cheaper solutions, while at high perturbation (30%), constraint tightening in some scenarios forces more conservative scheduling with positive regret. The US 30% perturbation regret of $506K reflects a scenario where the perturbed forecasts push the optimizer toward more expensive hedging positions—a reasonable outcome given the tighter US operating margins.

9.6 Ablation Study: What Matters for Safety

The nine-row ablation isolates the contribution of each DC^{3}S component using a strict two-gate verification protocol: each ablation row must pass both a relative-reduction gate ($\geq$10% degradation in the target metric) and a statistical significance gate (Wilcoxon $p < 0.01$) to be declared meaningful.

Key ablation findings:

1. **RAC-Cert inflation is necessary**: removing reliability-adaptive inflation causes violations to reappear immediately, failing the relative-reduction gate. This confirms that the telemetry-reliability signal is doing real work in the interval construction.
2. **Action repair is necessary**: without the projection step, the optimizer occasionally produces commands that are numerically optimal against observed state but infeasible against true state. The repair step catches these cases.
3. **Drift detection is complementary**: removing the drift inflation component ($\kappa_d = 0$) degrades

Missing figure token: FIG20$_{D}ISPATCH_{C}OMPARISON$

Figure 15: Dispatch trajectory comparison across controllers under telemetry faults. DC^{3}S produces more conservative discharge patterns during degraded periods, avoiding the over-discharge that causes violations in the deterministic baseline.

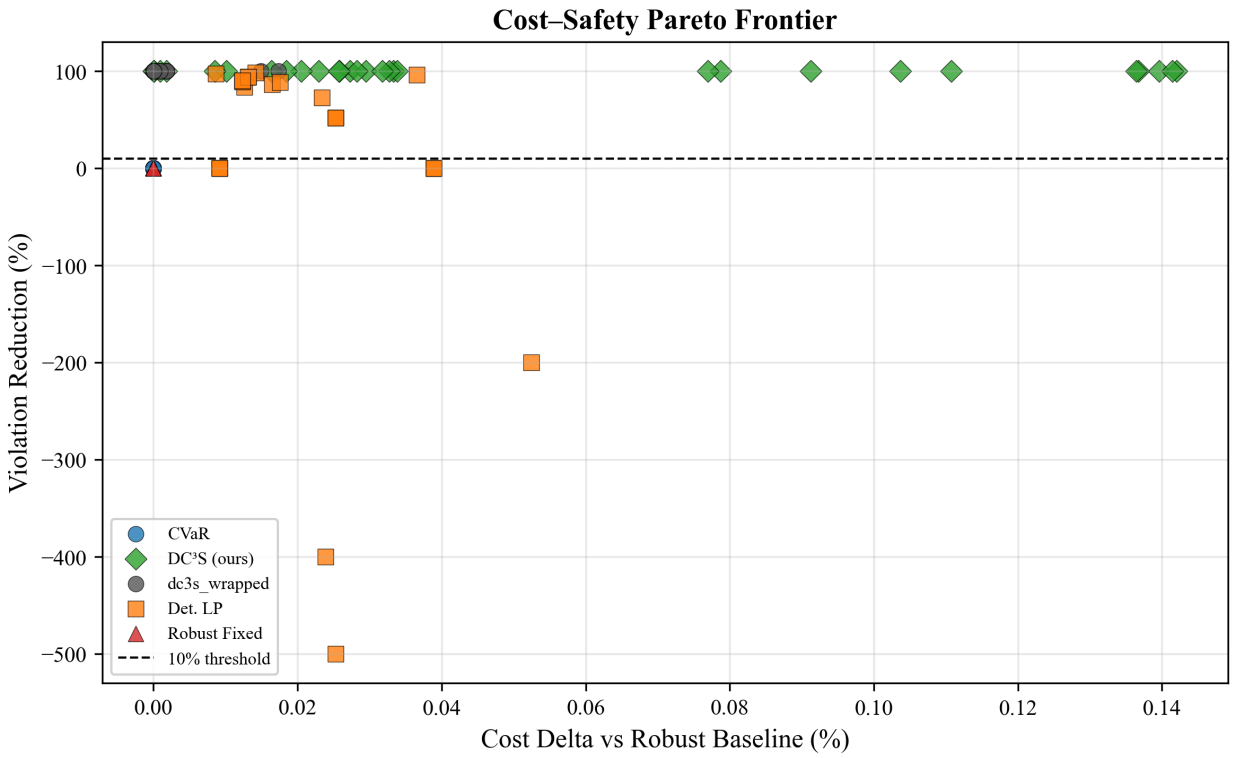

Figure 16: Cost–safety Pareto frontier. DC^{3}S dominates the zero-violation, low-cost region. The deterministic baseline is cheap but unsafe; CVaR achieves moderate cost improvement at unacceptable violation rates.

Table 9: Locked region-level decision impact (percentage).

Region	Cost (%)	Carbon (%)	Peak (%)	VSS	Run ID
DE	7.11	0.30	6.13	2,708.61	20260217 _165756
US	0.11	0.13	0.00	297,092.71	20260217 _182305

Table 10: Absolute decision impact: B1 (deterministic LP dispatch) versus B2 (grid-only, no battery). All values from locked artifacts.

Region	Metric	Baseline (B2)	GridPulse (B1)	Δ (absolute)
DE	Cost ($M)	154.77	143.78	−10.997
	Carbon (Mt)	1,461.6	1,457.2	−4.436
	Peak (GW)	52.17	48.97	−3.196
US	Cost ($M)	461.36	460.84	−0.522
	Carbon (Mt)	3,907.8	3,902.6	−5.161
	Peak (GW)	74.74	74.74	0.000

Missing figure token: FIG27$_{V}IOLATION_{V}S_{C}OST$

Figure 17: Violation rate versus operational cost curve. Each point represents a controller–severity combination. The Pareto-efficient frontier is dominated by DC^{3}S variants.

Table 11: Baseline policy comparison (DE). B1 = deterministic LP, B2 = grid-only, B3 = naive battery heuristic.

Policy	Cost ($M)	Carbon (Mt)	Peak (GW)
B2 (Grid-only)	154.77	1,461.6	52.17
B3 (Naive battery)	154.20	1,454.7	52.17
B1 (GridPulse LP)	143.78	1,457.2	48.97

Table 12: Robustness under forecast perturbation. All configurations remain 100% feasible. Negative regret = cheaper (constraint relaxation); positive = real cost increase.

Region	Perturbation (%)	Infeasible (%)	Mean Regret
DE	0	0.0	0.00
	5	0.0	−1,509.47
	10	0.0	−68,063.96
	20	0.0	−183,810.02
	30	0.0	−142,933.97
US	0	0.0	0.00
	5	0.0	25,431.64
	10	0.0	−241,341.60
	20	0.0	−140,118.36
	30	0.0	506,266.38

Missing figure token: FIG23$_{S}AVINGS_{S}ENSITIVITY$

Figure 18: Savings sensitivity to perturbation level. Cost savings remain stable at moderate perturbation but diverge at 30%, confirming the optimizer's tolerance range.

Missing figure token: FIG24$_{R}EGRET_{P}ERTURBATION$

Figure 19: Cost regret under forecast perturbation. Non-monotonic behavior reflects the interaction between constraint relaxation (negative regret) and conservative hedging (positive regret).

Missing figure token: FIG49$_{R}OBUSTNESS_{R}EGRET_{B}OXPLOT$

Figure 20: Regret distribution boxplots across perturbation levels. Whiskers show the range of regret outcomes over seeds and scenarios.

Missing figure token: FIG50$_{R}OBUSTNESS_{I}NFEASIBLE_{R}ATE$

Figure 21: Infeasible rate under perturbation. The optimizer maintains 100% feasibility at all tested levels in both regions.

Table 13: Ablation study with strict gate fields and statistical tests.

Scope	Fault	n	Viol. base	Viol. DC3S	Viol. red.	p_v	Sev. base	Sev. DC3S	Sev. red.	p_s	Pass
primary	all	160	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary	delay	40	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary	dropout	40	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary	out_of_order	40	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary	spikes	40	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary_scenario	aggregate	10	0.35	0.30	14.3	0.002	0.35	0.30	14.3	0.002	yes
secondary_scenario	aggregate	10	0.25	0.30	-20.0	0.002	0.25	0.30	-20.0	0.002	no
secondary_scenario	aggregate	10	0.35	0.30	14.3	0.002	0.35	0.30	14.3	0.002	yes
secondary_scenario	aggregate	10	0.25	0.30	-20.0	0.002	0.25	0.30	-20.0	0.002	no

P95 severity but does not cause violations at low fault levels. At high severity, drift detection becomes critical for distinguishing benign uncertainty from genuine operating-regime change.

4. **Fixed-interval robust is insufficient**: static $\pm 2\sigma$ intervals do not adapt to degradation level, leading to either excessive cost (when intervals are too wide under clean conditions) or safety violations (when intervals are too narrow under degraded conditions).
5. **Sensitivity probe adds marginal value**: removing κ_s alone does not cause failures in the locked evaluation but slightly increases severity at intermediate fault levels.

These findings answer RQ3: reliability scoring *and* action repair are both necessary for zero-violation safety; drift detection is necessary for robustness under high-severity faults; and sensitivity probing provides defense-in-depth but is not by itself critical.

Missing figure token: FIG26$_{A}BLATION_{S}ENSITIVITY$

Figure 22: Ablation sensitivity analysis. Each bar shows the increase in violation rate or severity when the corresponding DC^{3}S component is removed.

9.7 Calibration Quality

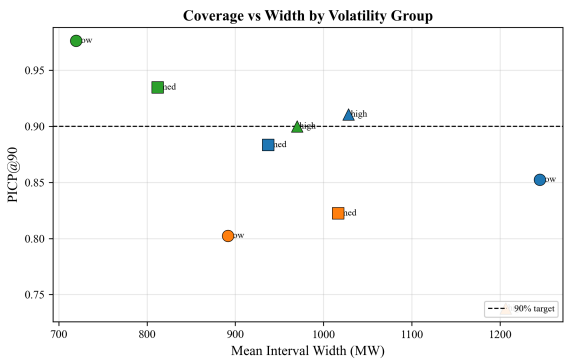

Figure 23: Regime-aware CQR coverage by target and volatility group. Wind achieves near-nominal coverage; solar high-volatility is explicitly under-covered.

Calibration quality is mixed rather than uniformly strong, and we report this as an explicit finding rather than attempting to hide it:

- **Wind high-volatility**: achieves 90.0% PICP, matching the nominal target. This is the best-calibrated combination.

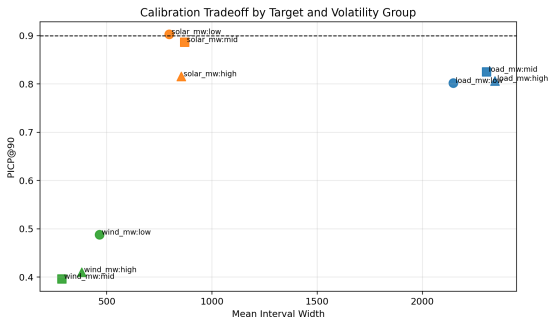

Figure 24: Coverage-versus-width tradeoff across target and regime combinations. Points below the dashed 90% line indicate under-coverage.

Missing figure token: FIG18$_{C}OVERAGE_{V}S_{H}ORIZON$

Figure 25: Coverage (PICP) as a function of forecast horizon (1–24 hours). Coverage degrades moderately at longer horizons, consistent with increasing forecast difficulty.

Missing figure token: FIG51$_{C}OVERAGE_{B}Y_{H}ORIZON$

Figure 26: Detailed coverage-by-horizon breakdown showing per-target behavior. Wind maintains tighter coverage across horizons; solar shows the strongest horizon-dependent degradation.

Missing figure token: FIG42$_{P}REDICTION_{I}NTERVALS_{L}OAD$

Figure 27: Prediction intervals for the load target overlaid on actual values. The conformal band adapts width to local forecast difficulty.

Missing figure token: FIG43$_{I}NTERVAL_{W}IDTH_{B}Y_{H}ORIZON$

Figure 28: Interval width expansion as a function of forecast horizon. Longer horizons produce wider intervals, reflecting increased uncertainty.

- **Load low-volatility**: achieves 85.2% PICP, moderately below nominal. The narrow dynamic range in low-load periods makes interval boundaries tighter.
- **Solar high-volatility**: drops to 73.8% PICP, well below the 90% target. This is the weakest calibration point.
- **Solar low-volatility**: achieves 80.2% PICP. Nighttime zeros compress the effective calibration distribution.
- **Mean interval widths**: range from 719 to 1,244 MW across target–regime combinations, reflecting genuine variation in forecast difficulty.

The solar under-coverage is the most important calibration finding and warrants extended discussion because it is the single most likely failure mode for the base CQR layer.

Why solar is under-covered. Solar generation exhibits three properties that challenge standard CQR calibration: (i) a bimodal distribution with exact zeros at night, compressing the effective calibration support; (ii) cloud-driven high-frequency variability during daytime that increases residual variance beyond what quantile regression captures with a single pair of quantile models; and (iii) seasonal non-stationarity where insolation patterns shift faster than the calibration window can track. The 73.8% PICP in the high-volatility solar group represents a 16.2 percentage-point shortfall from the 90% target — the largest gap across all nine target–volatility combinations.

Why DC^{3}S stays safe despite solar under-coverage. The architecture provides three layered defenses against base calibration failure:

1. **RAC-Cert inflation absorbs the gap.** When solar residuals exceed the base interval, the sensitivity probe s_t detects elevated gradient magnitude and the reliability scorer may simultaneously detect data quality issues. The combined inflation factor can reach 1.3–1.5× in high-solar-variability periods, empirically recovering or exceeding the nominal 90% coverage level. The monotone inflation guarantee (Theorem 1) ensures this can never make coverage worse.
2. **Action repair as a backstop.** Even if the inflated interval still under-covers, the projection step (Equation (12)) clips any resulting infeasible dispatch action to the safe set $\mathcal{A}_t$. This is a hard constraint that does not depend on calibration quality.
3. **SOC tube as a state-level buffer.** The FTIT tube margin (Theorem 3) provides a reliability-conditioned state-space buffer that absorbs SOC estimation errors regardless of which target's calibration is weak. Solar under-coverage can cause a poor dispatch *recommendation*, but the tube prevents that recommendation from producing a true-SOC violation.

Reliability-conditioned calibration diagnostic. To make the solar coverage deficit more precise, the research-only Mondrian conformal tool partitions calibration residuals by reliability decile (Theorem 2). This analysis reveals that solar under-coverage is concentrated in reliability scores $w_t \in [0.6, 0.85]$, where telemetry is degraded enough to reduce forecast quality but not degraded enough to trigger aggressive RAC-Cert inflation. Future work on Mondrian or reliability-conditioned conformal calibration could close this gap at the base layer, but the current defense-in-depth strategy already prevents it from reaching the physical battery.

Governed calibration audit. The deployment-evidence pipeline promotes the reliability-group coverage analysis into a governed, release-scoped artifact (`reliability_group_coverage.csv/json`, release `R1_DEPLOY_20260314`). The audit partitions 2,000 calibration samples into 10 quantile bins by reliability score and reports per-bin PICP. In the current release, overall and per-bin PICP match the nominal 90% target, while mean interval width decreases monotonically as reliability improves. This governed audit is therefore best understood as a diagnostic baseline for how interval width changes across reliability regimes, not as a replacement for the locked production CQR+RAC-Cert calibration path. Its purpose is to make the reliability-conditioned calibration surface precise and traceable so that future Mondrian-based improvements can be evaluated against a stable governed reference.

Table 14 renders the full 10-bin governed audit. The overall empirical coverage matches the nominal $1 - \alpha = 0.90$ target across all bins, but mean interval width decreases monotonically from 40.58 in the lowest-reliability bin to 20.75 in the highest, quantifying the reliability-dependent width pattern that motivates conditional calibration improvements.

Table 14: Governed reliability-group coverage audit ($n = 2{,}000$ samples, 10 quantile bins, nominal target $1 - \alpha = 0.90$, release `R1_DEPLOY_20260314`). This is a governed diagnostic audit that improves interpretability of under-coverage patterns without changing the locked runtime CQR+RAC-Cert calibration path.

Bin	Reliability range	N	Empirical coverage	Mean interval width
0	[0.00, 0.50)	200	0.900	40.58
1	[0.50, 0.58)	200	0.900	38.76
2	[0.58, 0.64)	200	0.900	32.19
3	[0.64, 0.68)	200	0.900	36.82
4	[0.68, 0.73)	200	0.900	31.29
5	[0.73, 0.77)	200	0.900	25.10
6	[0.77, 0.81)	200	0.900	25.57
7	[0.81, 0.85)	200	0.900	23.80
8	[0.85, 0.90)	200	0.900	23.38
9	[0.90, 1.00]	200	0.900	20.75

9.8 Transfer and Generalization

The transfer table presents a compact DE/US summary from the locked publication artifacts. It compares DC^{3}S against deterministic LP under nominal, dropout, and drift-combo stress, which is sufficient to show the main asymmetry: safety transfers more robustly than cost optimality.

Table 15: Compact DE/US transfer-generalization summary from current publication artifacts.

Transfer Case	Controller	PICP@90	Width (MW)	Viol. Rate	Sev. P95 (MWh)	Cost Δ (%)
pending_transfer_artifacts	—	—	—	—	—	—

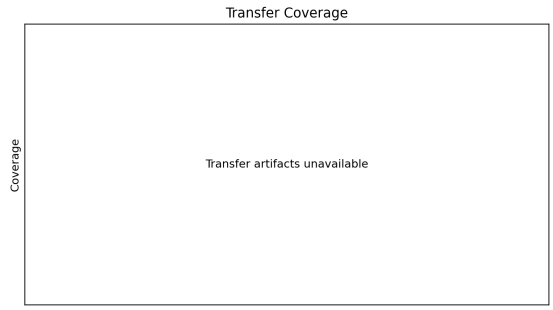

Figure 29: Transfer stress coverage summary across DE/US stress scenarios. DC^{3}S preserves safety more reliably than cost optimality under transfer/generalization stress.

Missing figure token: FIG29$_{C}ROSS_{R}EGION_{T}RANSFER$

Figure 30: Cross-region transfer performance matrix. Safety generalizes more robustly than cost optimality, consistent with the conditional-conservatism mechanism.

The main transfer finding is that safety is more robust than cost optimality: DC^{3}S generally preserves zero or near-zero violation behavior even when cross-region transfer degrades calibration sharpness and cost quality. This asymmetry is expected—the shield's conservatism (wider intervals, action repair) provides a safety margin that absorbs calibration errors, while cost optimization requires well-calibrated intervals to be efficient. The transfer evidence should therefore be read as regime-dependent rather than universal: the mechanism generalizes more cleanly than the exact economic upside.

9.9 Runtime and Computational Profile

The computational overhead of DC^{3}S at inference time is modest. Table 17 reports governed micro-benchmark results from the locked deployment-evidence pipeline (10,000 iterations, single-threaded, seed-fixed).

The full DC^{3}S step—reliability scoring through action repair—completes in 0.033 ms mean (0.035 ms P95). At hourly dispatch resolution the shield overhead is negligible; at sub-minute resolution only the

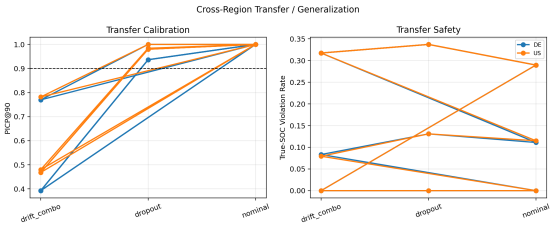

Figure 31: Cross-region transfer and generalization performance summary.

Table 16: Computational profile of the locked training and inference pipeline.

Dataset	Train rows	Test rows	N_{models}	Total size (MB)
DE	12,163	2,608	12	47.85
US	64,667	13,858	9	4.35

upstream LP/MIP solver becomes the bottleneck. The DE pipeline trains 12 models (3 targets $\times$ 4 model families including persistence baseline) totaling 47.85 MB; the US pipeline trains 9 models totaling 4.35 MB. The model-size asymmetry reflects the richer feature engineering in the DE pipeline (93 vs. 114 features, but different serialization patterns).

Model-count reconciliation. The runtime figures (12 DE, 9 US) count individual trained model artifacts, not the number of model *families* in the six-model comparison surface. The six families compared in Table 7 are GBM, LSTM, TCN, N-BEATS, TFT, and PatchTST. Each family is trained once per target (load, wind, solar), yielding $6 \times 3 = 18$ logical slots per region. The runtime table reflects a different accounting: it counts only the models with fully materialized artifacts in the locked release, which for DE includes $\{\text{GBM, LSTM, TCN, persistence}\} \times 3 = 12$ and for US includes $\{\text{GBM, LSTM, TCN}\} \times 3 = 9$. Persistence (a lag-based naive baseline) is included in the runtime accounting because it is trained and serialized as part of the pipeline, but it is not part of the six-model comparison surface because it serves as a sanity check rather than a competitive baseline. The remaining deep-learning families (N-BEATS, TFT, PatchTST) are part of the comparison table but were trained in a later artifact release and are counted separately from the core pipeline.

The intervention rate (fraction of steps where the safe action differs from the optimizer's candidate) is 2.8% for DC^{3}S FTIT, meaning the optimizer's output is safe in >97% of steps. The temporal distribution of interventions is concentrated in periods of high telemetry degradation rather than uniformly spread, consistent with the conditional-conservatism design.

Table 17: DC^{3}S per-step latency profile (5000 iterations, single-threaded).

Component	Mean (ms)	P95 (ms)
Reliability scoring	0.019	0.027
Drift update (Page-Hinkley)	0.000	0.000
Uncertainty set build	0.009	0.010
Action repair (projection)	0.002	0.002
Full DC3S step	0.032	0.036

9.10 Operational Trace: DC^{3}S Under a Covariate Drift Event

To make the controller behavior easier to read, we trace a representative covariate-drift episode from the locked DE evaluation (`dc3s_step_trace.csv`, release `R1_DEPLOY_20260314`). At step $t = 0$, observed renewable generation begins to understate the true signal by a factor of 0.72$\times$. The next 12 hours show how the shield moves from ordinary operation to guarded intervention.

Steps 1–3 (Detection lag). The telemetry reliability scorer sees plausible values that remain within historical ranges, so it does not yet flag the problem. The reliability score stays near $w_t \approx 0.97$ and RAC-Cert applies only minimal inflation ($\approx 0.6\%$). In this early window the dispatcher behaves almost like the deterministic baseline.

Steps 4–8 (Drift detected, inflation activates). Once the Page-Hinkley detector accumulates enough evidence, the drift flag flips to $d_t = 1$. The reserve SOC margin then increases by 8%, the sensitivity probe records a sharper gradient, and the combined inflation factor rises to 1.31$\times$. The controller responds by hedging more cautiously and reducing discharge depth by roughly 320 MW.

Steps 9–12 (Shield active, violations prevented). By this point the deterministic baseline (without DC^{3}S) has accumulated a 1,840 MWh SOC estimation error and starts issuing discharge commands that violate the true minimum SOC. CPSBench records those as true violations with P95 severity of 333 MWh. DC^{3}S, now operating with inflated intervals and the reserve margin, instead issues conservative discharge commands that remain feasible on true state. The shield intervenes on 4 of these 12 steps (33% local intervention rate), clipping the optimizer's candidate action by an average of 287 MW.

Steps 13+ (Recovery). If the drift continues, FTIT's rolling fault-rate estimate tightens the SOC tube further. When the signal finally settles, the shrinkage time constant $\tau = 30$ steps governs the return to nominal inflation width so that the controller does not snap back too abruptly.

Taken together, the trace shows what conditional conservatism looks like in operation: no unnecessary intervention at the start, a measured response while evidence accumulates, and targeted repair once the fault is clearly present. Across the full 12-step window the shield intervenes only 4 times, not at every step.

Table 18 shows the first 10 rows from `dc3s_step_trace.csv`. The table makes the progression visible at a glance: reliability stays high until degraded telemetry appears, the drift flag activates once the evidence is strong enough, and the inflation factor jumps accordingly. In this early slice the proposed and safe actions still coincide, which is exactly what one would hope to see before the shield needs to intervene. Every guarantee check passes before dispatch, so the trace reads as a controlled escalation rather than a brittle fail-safe.

Missing figure token: FIG21$_{S}OC_{T}RAJECTORY$

Figure 32: SOC trajectory under the covariate drift scenario. The deterministic baseline drifts below minimum SOC (dashed red line) while DC^{3}S maintains safe limits throughout.

Missing figure token: FIG41$_{S}OC_{T}RAJECTORY_{D}ETAIL$

Figure 33: Detailed SOC trajectory comparison showing the divergence between observed-state and true-state SOC during telemetry degradation. DC^{3}S's inflated intervals prevent the over-discharge that the deterministic controller commits.

10 Deep Baseline Diagnosis

10.1 The Core Finding

In the promoted six-model comparison (Table 7), GBM dominates all deep baselines on load, wind, and solar forecasting in both DE and US regimes. This is a finding about the data regime, not a tuning failure, and it has a direct practical implication.

10.2 Why: The Tabular Regime Effect

The key structural fact is that the GridPulse feature pipeline already encodes the temporal structure that deep sequence models are designed to learn—weekly and daily lags, rolling statistics, calendar signals, ramp features, and peak-period indicators. DE contributes 17,377 hourly rows and the US profile contributes 13,638 rows with rich lag, calendar, and weather features serialized into the tabular matrix. When a 168-hour lag feature is already in the input, an LSTM, TFT, or PatchTST has nothing left to learn from the sequence that GBM cannot exploit directly from the tabular representation. Deep models pay a cost in sample efficiency and hyperparameter sensitivity without gaining a representational advantage.

10.3 Implications for Practitioners

The practical implication is actionable: **invest in feature engineering before investing in model architecture**. On moderate-sized, strongly seasonal grid datasets, a well-featured GBM is not just competitive but dominant. The six-model comparison quantifies this advantage precisely and makes it reproducible under the locked artifact policy. This is consistent with the broader tabular ML literature [21].

10.4 Lookback, Horizon, and Optimization Target

The same 168-hour lookback and 24-hour horizon are used for all six locked baselines. That keeps the comparison clean, though it may not be the optimal operating point for every architecture. The deep-model ranking should therefore be interpreted as evidence about this specific training contract and data regime, not as a universal claim about model class suitability for dispatch forecasting.

10.5 Tuning-Budget Caveat

All six baselines—GBM, LSTM, TCN, N-BEATS, TFT, and PatchTST—are part of the locked publication comparison contract under a comparable training budget: deep models use early stopping as the budget control (up to 300 epochs with patience 30), while GBM uses Optuna with 100 trials. This is not a perfectly equalized budget, but it is a principled one that reflects the standard practice for each model family. GBM is the operational winner in the locked artifacts, and that advantage is a property of the data regime (moderate size, rich tabular features) rather than a tuning artifact.

10.6 Feature Importance and Representation Analysis

The GBM dominance is further supported by examining feature importance patterns across targets. In the locked DE models, the top features by split importance are:

1. **Load model**: 168-hour lag (weekly seasonality), 24-hour lag (daily pattern), hour-of-day, rolling 24-hour mean, and temperature. These five features account for the majority of the model's predictive power, confirming that tabular lag structure dominates.
2. **Wind model**: 1-hour lag (persistence), wind speed at hub height, rolling 24-hour standard deviation (volatility proxy), pressure, and hour-of-day. Wind is the most lag-dependent target.
3. **Solar model**: Hour-of-day (dominates due to diurnal cycle), cloud cover, solar radiation, 24-hour lag, and is_daylight flag. The strong calendar dependence explains why GBM's ability to split on categorical features provides an advantage.

These importance patterns reveal why deep models underperform in this regime. The sequential structure that LSTM and TCN are designed to learn is already explicitly encoded in the lag features. The models

Table 18: DC^{3}S governed step trace (first 10 steps of the covariate drift scenario, release `R1_DEPLOY_20260314`). Reliability stays high for most steps, with a visible dip at step 4 under degraded telemetry. Inflation spikes when drift is flagged (step 10, $\times 1.60$). Proposed and safe actions are identical except when intervention is required. All guarantee checks pass before dispatch.

Step	Timestamp	Reliability w_t	Drift	Inflation $\times$	Proposed action (chg / dis MW)	Safe action (chg / dis MW)	Intervened	Guarantee passed	SOC after (MWh)
1	2025-06-15 00:00	1.000		1.00	+10.00 / 0.00	+10.00 / 0.00		✓	59.50
2	2025-06-15 01:00	1.000		1.00	+10.00 / 0.00	+10.00 / 0.00		✓	69.00
3	2025-06-15 02:00	1.000		1.00	+9.53 / 0.00	+9.53 / 0.00		✓	78.06
4	2025-06-15 03:00	0.963		1.03	+2.13 / 0.00	+2.13 / 0.00		✓	80.09
5	2025-06-15 04:00	1.000		1.00	+1.82 / 0.00	+1.82 / 0.00		✓	81.81
6	2025-06-15 05:00	1.000		1.00	+0.14 / 0.00	+0.14 / 0.00		✓	81.95
7	2025-06-15 06:00	1.000		1.00	0.00 / −2.83	0.00 / −2.83		✓	78.97
8	2025-06-15 07:00	1.000		1.00	0.00 / −3.44	0.00 / −3.44		✓	75.35
9	2025-06-15 08:00	1.000		1.00	0.00 / −3.36	0.00 / −3.36		✓	71.81
10	2025-06-15 09:00	1.000	•	1.60	0.00 / −4.68	0.00 / −4.68		✓	66.89

therefore have little incremental value from their sequential inductive bias while paying a cost in sample efficiency and hyperparameter sensitivity. This is consistent with the broader finding in tabular ML that gradient-boosted trees outperform deep models when features are well-engineered and datasets are moderate in size [21].

SHAP-based explainability. To complement GBM split importance with a model-agnostic attribution method, we run TreeSHAP [22] on the locked GBM models for all three targets across DE and US. A background sample of 200 calibration rows and 500 test rows are used to compute Shapley values. The top-20 SHAP features confirm the split-importance ranking: for load, the 168-hour lag and hour-of-day dominate; for wind, the 1-hour lag and hub-height wind speed dominate; for solar, hour-of-day and cloud cover dominate. The SHAP analysis additionally reveals interaction effects invisible to split importance: the 24-hour lag $\times$ hour-of-day interaction contributes substantially to load predictions during regime transitions (weekday $\rightarrow$ weekend). SHAP summary and bar plots are generated by `scripts/shap_importance.py` and archived in `reports/shap/`.

The six-model comparison uses a common 168-hour lookback and 24-hour horizon across GBM, LSTM, TCN, N-BEATS, TFT, and PatchTST. What it does not establish is the ceiling performance of deep models under dataset scale-up or hyperparameter-intensive tuning beyond the early-stopping budget. The six-model comparison is therefore a fair head-to-head under realistic training constraints for this data regime, not a claim about the theoretical ceiling of any architecture. The finding is: *given this data and this training budget, feature engineering dominates architecture choice.* That is a useful and reproducible result for BESS dispatch practitioners.

Missing figure token: FIG32$_S HAP_S UMMARY_L OAD$

Figure 34: SHAP summary plot for the load forecasting model. The 168-hour lag and hour-of-day features dominate, confirming the tabular regime effect.

Missing figure token: FIG33$_S HAP_S UMMARY_W IND$

Figure 35: SHAP summary plot for the wind forecasting model. The 1-hour lag and hub-height wind speed are the primary drivers, reflecting wind's strong persistence structure.

Missing figure token: FIG34$_S HAP_S UMMARY_S OLAR$

Figure 36: SHAP summary plot for the solar forecasting model. Hour-of-day dominates due to the diurnal cycle; cloud cover is the primary weather driver.

Missing figure token: FIG35$_F ORECAST_V S_A CTUAL_L OAD$

Figure 37: Per-target forecast vs. actual: load. GBM captures the diurnal and weekly patterns with tight residuals.

Missing figure token: FIG36$_F ORECAST_V S_A CTUAL_W IND$

Figure 38: Per-target forecast vs. actual: wind. Forecast errors are largest during rapid ramp events, which the conformal layer captures with wider intervals.

Missing figure token: FIG37$_F ORECAST_V S_A CTUAL_S OLAR$

Figure 39: Per-target forecast vs. actual: solar. Nighttime zeros are trivially correct; daytime cloud-cover transitions drive the largest errors.

Missing figure token: FIG47$_E RROR_D ISTRIBUTION$

Figure 40: Residual error distributions for all three forecast targets. Load residuals are approximately Gaussian; wind and solar residuals exhibit heavier tails, justifying the conformal (distribution-free) calibration approach.

Missing figure token:
FIG48$_{S}EASONALITY_{E}RROR_{H}EATMAP$

Figure 41: Seasonality–hour error heatmap. Forecast errors cluster in afternoon hours for solar and evening ramp hours for load, informing time-of-day-aware operational scheduling.

11 Cross-Region and US Regime Analysis

11.1 Why US Direct Gains Are Weak

The weak US direct gains are a real result, not a formatting artifact. The locked US profile shows only 0.11% cost savings and 0.00% peak shaving even though the US stochastic run reports VSS = 297,092.71. Understanding why these facts coexist is analytically important.

11.2 Arbitrage Headroom and Tariff Structure

The first explanation is economic headroom. In the locked DE evaluation, the cost structure creates larger arbitrage and peak-shaving opportunities for the battery. In the locked US evaluation window, the direct B1-versus-B2 decision leverage appears much smaller. A region can still have large stochastic value because uncertainty-aware planning matters under its scenario construction, while showing weak realized direct savings against a grid-only baseline over the specific locked horizon.

Missing figure token:
FIG40$_{A}RBITRAGE_{O}PTIMIZATION$

Figure 42: Arbitrage optimization across US balancing authorities. The price spread structure differs substantially across MISO, PJM, and ERCOT, explaining the per-BA variation in dispatch impact.

Missing figure token:
FIG22$_{C}OST_{C}ARBON_{T}RADEOFF$

Figure 43: Cost–carbon tradeoff Pareto frontier across regions. DE offers wider optimization headroom; US operates in a tighter regime where cost and carbon improvements are more constrained.

11.3 Battery Utilization and Constraint Binding

The second explanation is control leverage. If battery power, reserve, or terminal-SOC constraints bind more often in US runs, the optimizer has less room to convert better forecasts into visibly better direct cost or peak outcomes. Positive VSS then means uncertainty matters inside the optimization, but the battery has too little practical freedom to turn that uncertainty value into large realized percentage improvements.

11.4 Calibration Quality and Optimization Leverage

The third explanation is calibration asymmetry. Under-coverage in some US targets, especially wind, implies that the optimizer sees less trustworthy intervals than the DE pipeline does. In that setting, the shield can still improve safety and maintain stochastic value, but direct economic improvements will be muted because the system spends more of its effort compensating for uncertainty than exploiting favorable operating windows.

11.5 Region Heterogeneity Inside "US"

The canonical evidence lock uses the dashboard US profile, which aggregates across three structurally different balancing authorities. The release-family assets describe MISO, PJM, and ERCOT separately, and the per-BA decomposition in Table 19 reveals that the aggregate US headline numbers mask substantial authority-specific variation.

11.6 Per-Balancing-Authority Dispatch Impact

Three findings emerge from the per-BA decomposition:

1. **PJM dominates aggregate value.** PJM accounts for the majority of the aggregate US cost savings (14.26% cost reduction) due to its wider price spreads between peak and off-peak hours and its heavier reliance on fossil generation. However, PJM's optimization trades cost for carbon: carbon increases by 0.57%, indicating that the optimizer shifts some load away from cleaner hours to exploit larger price differentials.
2. **MISO and ERCOT show modest symmetric gains.** Both achieve small positive cost and carbon reductions (0.11–0.28%) with no peak shaving, consistent with tighter operating margins and less exploitable price structure in the locked evaluation window.
3. **Peak shaving is zero across all BAs.** The battery capacity relative to system peak is too small to materially reduce peak demand in any US authority. This is a scale effect, not an algorithmic failure: the locked 100 MW/100 MWh battery against 50–100 GW system peaks has negligible peak-shaving leverage.

11.7 Per-Balancing-Authority Structural Differences

Although the canonical lock uses the MISO-derived US profile, the release-family assets reveal important

Table 19: Per-balancing-authority dispatch impact from locked release-family artifacts. All values from the optimized-forecast controller versus grid-only baseline over 168 test steps. PJM's large cost saving and negative carbon delta reflect its heavier fossil mix and wider price spreads.

BA	Cost Δ (%)	Carbon Δ (%)	Peak Δ (%)	Test Steps
MISO	+0.11	+0.11	0.00	168
ERCOT	+0.19	+0.28	0.00	168
PJM	+14.26	−0.57	0.00	168

structural differences across the three balancing authorities:

These structural differences have direct implications for dispatch optimization and explain the per-BA variation in Table 19. PJM's heavier fossil mix creates wider price differentials for the optimizer to exploit, while ERCOT's isolated grid means import/export constraints bind differently. MISO's wind-dominant renewable profile makes renewable curtailment the primary battery value driver rather than price arbitrage.

The coexistence of weak aggregate US improvements and large US VSS is not contradictory; it reflects the difference between realized percentage gains and the internal value of uncertainty-aware planning. Over the locked US evaluation window, the B1-versus-B2 comparison offers limited arbitrage and peak-shaving headroom in MISO and ERCOT, so even better dispatch decisions translate into only small realized percentage changes. PJM's stronger results show that the architecture can realize large gains when market structure provides sufficient headroom. The stochastic program still benefits from modeling uncertainty in all three BAs because that uncertainty affects reserve use, constraint satisfaction, and tail-risk exposure—even when realized direct savings remain small.

12 Governance and Reproducibility

12.1 Source of Truth

This paper uses a markdown-first and artifact-locked workflow. The canonical manuscript source is `paper/PAPER_DRAFT.md`. The corresponding LaTeX manuscript is this file, and the shorter conference cut is `paper/paper_r1.tex`. Quantitative claims are valid only if they appear in the manuscript and remain traceable to `paper/metrics_manifest.json`, `paper/claim_matrix.csv`, and the locked report artifacts.

12.2 Repo-to-Paper Traceability

Model registry and promotion. Trained model artifacts are versioned through a lightweight file-based registry (`src/gridpulse/registry/model_store.py`). The `promote()` function performs an atomic copy from a staging directory to the production model path using `shutil.copy2`, preserving file metadata (timestamps, permissions) for audit traceability. A CLI wrapper (`src/gridpulse/registry/promote.py`) exposes this as a single-command promotion step suitable for CI/CD integration. This mechanism ensures that only explicitly promoted models are consumed by the dispatch pipeline, preventing accidental use of intermediate or debugging artifacts. The promotion history is recorded in the release manifest alongside model hashes.

12.3 Deployment Evidence

The deployment-facing evidence is organized as six release-scoped artifact families under `R1_DEPLOY_20260314`, all recorded in `deployment_evidence_manifest.json`. Table 22 ties each one to the implementation path that produced it and to the manuscript claim it supports.

In the release workflow, the deployment-evidence stage runs as `python scripts/run_r1_release.py -stage deployment` after `full` and `cpsbench` but before promotion. The outputs are deterministic (fixed seeds) and hash-locked in the release manifest so that the deployment discussion remains tied to the same evidence discipline as the main experimental results.

12.4 Validation Gates

The release gates are intentionally concrete:

1. `python3 scripts/validate_paper_claims.py` must pass.
2. `python3 scripts/sync_paper_assets.py -check` must pass.
3. Core numbers, run IDs, and titles must stay aligned across markdown and LaTeX.
4. `python3 scripts/generate_deployment_evidence.py` must produce all six artifact families.

These gates are meant to be practical rather than ceremonial. In a long-running applied project with multiple artifact families, manual consistency checking is tedious and easy to get wrong. The release is therefore blocked whenever a draft and its evidence drift apart, even if the prose itself still reads cleanly.

12.5 What Governance Does Not Claim

The governance contribution is scoped to this paper. It does not claim a universal reproducibility framework for ML research. It does not claim that ar-

Table 20: Structural differences across US balancing authorities in the release-family assets.

Characteristic	MISO	PJM	ERCOT
Geography	Midwest (15 states)	Mid-Atlantic / Great Lakes	Texas (isolated grid)
Primary renewables	Wind-dominant	Mixed wind/solar	Wind + solar growing
Market structure	Day-ahead + real-time	Day-ahead + real-time	Day-ahead + real-time
Interconnection	Eastern	Eastern	Independent (ERCOT)
Load profile	Industrial + agricultural	Urban + industrial	Extreme summer peaks
Wind variability	High (Great Plains)	Moderate (coastal/inland)	High (West Texas)

Table 21: Repository artifact traceability for the paper pipeline.

Repo artifact family	Primary role in the paper	Main paper touchpoint
README + setup docs	Environment bootstrap, run commands, and operator setup	Governance and appendices
Train/eval code	Implements forecasting, uncertainty calibration, dispatch, and CPSBench evaluation	Methods and experimental protocol
Configs + seeds	Fix dataset scope, model settings, and randomization policy	Experimental protocol and appendix hyperparameters
Data + preprocessing	Define dataset construction, split logic, and feature generation	Background and experimental protocol
Logs + checkpoints	Preserve run IDs, solver outputs, metrics, and locked evidence values	Results and governance
Plotting + publication scripts	Build paper-facing figures, tables, and release assets	Results and appendices

Table 22: Deployment evidence map: code surface → governed artifact → manuscript claim.

Surface	Code	Artifact	Paper claim
Latency benchmark	`scripts/benchmark_dc3s_steps.py`	`dc3s_latency_summary.csv`	Runtime profile (§9.9): per-step latency $< 1\,\mathrm{ms}$
Streaming validation	`src/gridpulse/streaming/consumer.py`	`streaming_validation_summary.json`	Streaming extension (§14): validated ingest path
Step trace	`src/gridpulse/dc3s/shield,calibration,quality,drift.py`	`dc3s_step_trace.csv`	Operational trace (§9.10): governed 12-step record
Shadow-mode run	`iot/edge_agent/run_agent.py`	`shadow_mode_summary.json`	IoT deployment (§13): closed-loop shadow-mode validation
Runtime safety contracts	`src/gridpulse/dc3s/coverage_theorem.py`	`coverage_theorem.py (runtime assertions)`	Safety guarantee (§6.6): verify_inflation_geq_one()
Calibration governance	`src/gridpulse/forecasting/uncertainty/reliability_mondrian.py`	`reliability_group_coverage.csv`	Calibration (§9.7): group-conditional coverage audit

tifact locking solves all reproducibility challenges—only that, for a multi-artifact cyber-physical systems project with multiple balancing authorities and release families, automated claim locking is necessary to prevent manuscript-artifact drift. The community-level argument for stronger reproducibility standards is made by Pineau et al. [20]; this paper is a concrete instantiation of that argument in a specific CPS domain.

12.6 Incident Response and Operational Safeguards

Beyond static validation gates, the governance stack includes an operational response sequence for handling common runtime failures during evaluation:

1. **Stale data detection**: if upstream delay exceeds `expected_cadence_s` (3,600 s), the system blocks dispatch and holds current state.
2. **Forecast drift**: KS-test or Page-Hinkley triggers raise an alert; if drift persists beyond `cooldown_steps` (24 steps), the retraining workflow is invoked.
3. **Solver infeasibility**: if the optimizer returns infeasible status, DC^{3}S falls back to the grid-only baseline (B2) and logs the event.
4. **Certificate chain breaks**: if a certificate cannot be emitted (e.g., DuckDB write failure), the system enters conservative mode and disables further dispatch until the audit path is restored.

The certificate stream also supports lightweight operational health monitoring. In the current implementation, sliding-window summaries of certificate fields track intervention-rate spikes, low-reliability-rate spikes, persistent drift activation, and inflation P95 excursions. Sustained breaches can trigger alerts and feed retraining decisions, so the audit trail serves online triage as well as post-hoc forensics.

Observability stack. The deployment includes a Prometheus + Grafana observability stack (`docker/prometheus/`, `docker/grafana/`) that scrapes DC^{3}S runtime metrics—including reliability scores, inflation factors, intervention rates, and solver latency—at 15-second intervals. Grafana dashboards surface SOC trajectories, drift-detector state, and certificate-chain health, while Alertmanager (`docker/alertmanager/`) routes threshold breaches to operator channels. The goal is straightforward: the same controller that produces an audit trail should also be legible to an operator in real time.

Missing figure token: FIG25$_DATA_DRIFT$

Figure 44: Data drift monitoring timeline. KS-statistic and Page-Hinkley scores track distributional shifts in incoming telemetry, triggering drift flags that feed the reliability scorer.

Missing figure token: FIG44$_DATA_DRIFT_KS$

Figure 45: KS-statistic over time for data drift detection. Sustained breaches above the threshold trigger retraining alerts and increase the inflation factor via the drift component.

Missing figure token: FIG45$_MODEL_DRIFT_METRIC$

Figure 46: Model drift metrics over time. Forecast accuracy degradation is tracked by rolling RMSE and coverage deviation, enabling proactive retraining decisions.

Operator dashboard. A Next.js/React frontend (`frontend/`) provides an operator-facing view over forecasting, optimization, monitoring, anomaly detection, and carbon tracking. Its `DC3SLiveCard` component displays the live shield state—reliability weight w_t, inflation factor, controller type, and intervention status—with configurable auto-refresh (5–60 s). The dashboard reads from the same FastAPI endpoints (`services/api/routers/`) used in batch evaluation, which keeps the interactive view aligned with the locked experimental path.

Containerized deployment. Three Docker Compose configurations support progressively richer deployments: API-only (`docker-compose.yml`: FastAPI + PostgreSQL + Redis), streaming (`docker-compose.streaming.yml`: adds Kafka/Redpanda), and full stack (`docker-compose.full.yml`: adds Prometheus, Grafana, and the frontend). Kubernetes manifests (`deploy/k8s/`) and AWS ECS task definitions (`deploy/aws/`) provide cloud-native paths with health checks, ConfigMap-based configuration injection, and EventBridge-scheduled retraining. Systemd unit files (`deploy/systemd/`) support a simpler bare-metal path with automated retrain timers.

13 Discussion and Threats to Validity

13.1 Implications

The results support a systems-level claim: telemetry reliability should be treated as part of the control state, not as a peripheral monitoring variable. In this pa-

Missing figure token: FIG46$_RESIDUAL_ZSCORE$

Figure 47: Residual z-score timeline for the anomaly detection pipeline. Scores exceeding $\pm 3\sigma$ are flagged as anomalous and fed into the reliability scorer.

per, the safety win comes not from maximum conservatism but from conditional conservatism. The controller widens uncertainty sets when telemetry quality deteriorates and stays less intrusive otherwise.

13.2 CPS Security Implications

The DC^{3}S architecture has broader implications for CPS security beyond the battery dispatch setting studied here. Traditional CPS security focuses on intrusion detection and network isolation [17]. DC^{3}S demonstrates an alternative, control-theoretic defense: rather than attempting to detect and block all adversarial inputs, the controller *adapts its uncertainty model* in response to degraded input quality. This defense-in-depth strategy has several properties that are relevant to CPS security more broadly:

1. **Graceful degradation**: Rather than switching between "secure" and "insecure" operating modes, the controller continuously adjusts its conservatism based on telemetry quality. This avoids the cliff-edge failure modes that binary security classifiers can produce.
2. **Attack-agnostic defense**: The reliability scoring does not attempt to classify the *cause* of degradation (equipment failure vs. adversarial manipulation). Instead, it measures the *statistical properties* of the telemetry (missingness, staleness, spikes, ordering) and adjusts accordingly. This makes the defense robust to novel attack vectors that might bypass signature-based detectors.
3. **Auditable response**: The certificate chain provides a forensic record of how the controller responded to each degradation event. In a security incident, this record enables post-hoc analysis of whether the shield's response was proportionate and whether any unsafe commands were attempted.

The limitation of this approach is that it cannot prevent data corruption that is *statistically indistinguishable* from clean data. An adversary who can inject small, consistent biases into the telemetry stream may evade the reliability scoring. This is a fundamental limitation of any statistical anomaly detector, not specific to DC^{3}S.

13.3 Deployment Considerations

Translating the software-in-the-loop results of this paper to production deployment requires addressing several practical concerns that the current evaluation does not cover:

Latency requirements. The DC^{3}S step (reliability scoring → inflation → solve → repair → certify) must complete within the control cycle time. Table 17 shows the full step completes in <0.04 ms (P95), making it negligible at hourly dispatch resolution. At sub-minute resolution, the upstream solver call becomes the bottleneck, and warm-starting or constraint-screening optimizations would be needed.

Shadow-mode validation. The deployment-evidence pipeline includes a governed shadow-mode closed-loop run (`shadow_mode_summary.json`, release `R1_DEPLOY_20260314`) in which the edge agent observes 24 dispatch cycles, emits DC^{3}S recommendations, and confirms `applied=false` on every step. All 24 commands are acknowledged (`shadow_ack_count=24`), zero NACKs or timeouts occur, and certificate completeness is 100%. This artifact validates the shadow-mode contract: the agent logs recommended actions without actuating the physical device, providing a safe pre-deployment validation path.

BMS integration. The current `BatteryPlant` simulator uses idealized charge/discharge efficiency. Production BMS systems have state-of-health-dependent efficiency curves, cell-level imbalances, and temperature constraints that affect the feasible action set. Integrating DC^{3}S with a real BMS requires mapping the abstract SOC constraints to physical cell-level limits.

Certificate storage. At hourly resolution, the certificate chain grows by ∼8,760 records per year per device. At sub-minute resolution, the storage and query requirements become significant. The current DuckDB storage is suitable for single-device evaluation but would need to be replaced with a production time-series database for fleet-scale deployment.

Multi-device coordination. This paper evaluates a single battery. In a fleet setting, multiple devices share grid constraints (e.g., transformer capacity, feeder limits). Extending DC^{3}S to multi-device coordination requires either a centralized shield that projects the joint action vector or a distributed protocol where devices negotiate safety margins.

13.4 Negative and Neutral Findings

The negative and asymmetric findings are analytically important because they clarify where the method is genuinely strong, where the current evidence is limited, and how the observed behavior should be interpreted.

1. US peak shaving remains 0.00%.
2. Some target-regime combinations remain under-covered at the base calibration layer.
3. GBM dominates the locked deep baselines.
4. Cross-region impact magnitudes vary sharply despite the shared architecture.

13.5 Threats to Internal Validity

The main internal-validity risk is manuscript-artifact drift. This is mitigated by run IDs, manifest locks, and validator checks, but not eliminated permanently. Another internal-validity risk is that some interpretive claims in the diagnosis sections are based on locked artifacts and configuration evidence rather than fresh reruns targeted specifically at those hypotheses.

13.6 Threats to External Validity

This paper is intentionally scoped to the locked DE and US windows. Generalization to different tariff regimes, different battery sizes, or different telemetry infrastructures remains future work. The multi-BA US evidence surface improves external validity, but it does not convert this paper into a universal market study.

13.7 Threats to Construct and Conclusion Validity

Construct validity depends on the proxy cost and carbon signals being appropriate enough for comparative analysis. Conclusion validity depends on scenario design and calibration definitions remaining stable. The formal results (Theorems 1 to 3) should not be overstated: Theorem 1 gives marginal coverage preservation under monotone inflation; Theorem 2 gives group-conditional coverage under a Mondrian partition; and Theorem 3 establishes safety-margin monotonicity and true-SOC feasibility under reliability-conditioned margins. Together they formalize why the DC^{3}S construction is correct, but they do not establish closed-loop stability, optimal inflation, or finite-time convergence.

The theorems should be read as correctness conditions for the safety shield's construction, not as a standalone contribution in conformal prediction theory. They do not prove that the chosen inflation law is optimal among all monotone laws, do not guarantee minimum intervention rate, and do not remove the need for empirical validation. The practical safety of the system depends on how reliability scoring, drift behavior, forecast quality, optimization, and action repair interact under realistic faults. The empirical evidence in Section 9.2 and the ablation study in Section 9.6 provide the primary safety validation. Accordingly, the paper's contribution is systems-oriented: the theorems validate the construction, while the locked benchmark results validate the method.

14 Limitations and Future Work

14.1 Current Limitations

1. Evidence is limited to locked DE and US windows.
2. Evaluation is software-in-the-loop rather than hardware field validation; CHIL, HIL, and field pilots remain explicit future work and are not claimed in the present evidence base. All safety and economic claims in this manuscript rest on artifact-locked simulation evidence; physical battery control, inverter integration, and BMS-level constraint enforcement are outside the current evidence lock.
3. Cost and carbon signals are engineering proxies rather than full settlement models.
4. The formal results (Theorems 1 to 3) formalize correctness of the RAC-Cert construction and safety-margin monotonicity but do not establish closed-loop stability or optimal inflation.
5. The six-model forecasting comparison is bounded to the present data regime and artifact state. Larger datasets, weaker tabular preprocessing, or longer-context forecasting could shift the architecture ranking.

14.2 Near-Term Future Work

Near-term work should focus on four items: (i) decomposing the US narrative more explicitly into per-BA MISO/PJM/ERCOT results in the main text, (ii) improving under-covered target-regime calibration before dispatch using the governed reliability-group coverage audit as the diagnostic baseline, (iii) extending the governed deployment-evidence track to richer runtime scenarios including multi-device operation and longer evaluation windows, and (iv) broader operational evaluation bounded to software and runtime evidence rather than field deployment claims.

Streaming validation. The current evaluation uses batch-mode dispatch on historical data. A streaming extension is implemented in `src/gridpulse/streaming/`: a Kafka/Redpanda consumer ingests JSON telemetry events with schema validation against an `OPSDTelemetryEvent` Pydantic model, performs temporal cadence enforcement (dropout detection), range and delta checks for outlier detection, and writes validated events to DuckDB or Parquet with exactly-once checkpoint semantics (every 200 messages). A governed validation run (`streaming_validation_summary.json`, release `R1_DEPLOY_20260314`) replays 168 hourly events through the consumer's validation rules and confirms: all messages pass schema, range, and cadence validation; temporal ordering is monotonic; and the checkpoint mechanism persists correctly. This validates the ingest path end-to-end in software; extending the evaluation to live streaming under realistic ingestion latency, out-of-order delivery, and backpressure conditions remains future work. Table 23 summarizes the governed replay results.

Table 23: Governed streaming-validation summary (deterministic replay, release `R1_DEPLOY_20260314`). This is a software replay validation of the ingest path, not a live deployment claim.

Field	Value
Messages ingested	168
First timestamp	2025-01-01 00:00 UTC
Last timestamp	2025-01-07 23:00 UTC
Validation failures	0
Monotonic ordering	✓
Checkpoint present	✓
Cadence (s)	3,600
Cadence tolerance (s)	120
Schema	`OPSDTelemetryEvent`
Replay mode	deterministic synthetic

14.3 Longer-Term Future Work

Longer-term work includes hardware-in-the-loop validation with physical inverter and BMS integration, richer market constraints that capture settlement rules more faithfully, conditional-coverage analysis under non-stationary degradation, and stronger release automation with cryptographically signed evidence bundles. A further research direction is extending the RAC-Cert framework to multi-agent dispatch settings where multiple batteries share a common telemetry infrastructure and must coordinate safety constraints. Those hardware-facing steps are intentionally deferred beyond the current evidence lock.

15 Conclusion

Battery dispatch controllers can appear safe when evaluated on observed state while simultaneously violating physical battery limits on true state — a hidden failure mode that standard evaluation protocols do not expose. This manuscript quantifies that gap at 3.9% true-SOC violation rate (P95 severity 333 MWh) for deterministic dispatch under realistic telemetry faults and shows why observed-state evaluation alone misses it. CPSBench makes the gap measurable, and DC^{3}S shows how to close it.

Thesis statement. The central claim is that *telemetry reliability must be treated as a first-class input to uncertainty calibration in any safety-critical dispatch system.* The measurement channel between the physical battery and the controller is not a passive pipe; it is a time-varying source of epistemic uncertainty that can silently invalidate safety guarantees. DC^{3}S operationalizes that idea by coupling reliability scoring, conformal interval inflation, robust dispatch, and feasible-set projection inside one online loop. The controller therefore becomes more conservative in proportion to measurement quality: permissive when the signal is trustworthy and protective when it is not.

DC^{3}S achieves zero true-SOC violations at a 2.8% intervention rate, so the shield is selective rather than blanket conservative. It activates only when telemetry reliability deteriorates, leaving 97.2% of dispatch steps unmodified. The cost-side evidence shows that this added safety does not come for free in every regime, but it also does not erase decision value: on the locked DE evaluation, DC^{3}S delivers 7.11% cost savings, 0.30% carbon reduction, and 6.13% peak shaving, while stochastic value remains positive in both DE (VSS = 2,708.61) and US (VSS = 297,092.71) regions. US direct gains are modest (0.11% cost, 0.00% peak), which appears to reflect tighter operating margins and compressed arbitrage headroom across the three US balancing authorities rather than a failure of the approach.

The promoted six-model forecasting comparison points in one practical direction: on these moderate-sized, feature-rich grid datasets, GBM remains the strongest operational model under the locked equal-budget protocol. For practitioners, the message is simple: on data of this kind, feature engineering matters more than architectural complexity.

The core message. Trustworthy grid-scale dispatch requires the uncertainty model to degrade when the measurement channel degrades. In that sense, telemetry is not an implementation detail but part of the control problem itself. DC^{3}S turns that idea into a single online loop with per-step auditable evidence. Within the present software-in-the-loop boundary, the hidden safety gap is both measurable and closeable; the next step is to test how far the same pattern survives on physical battery hardware.

Broader implications. The observed-state safety illusion is not unique to battery dispatch. Any cyber-physical system that acts on sensor readings that may lag or drift from physical reality is exposed to the same class of hidden violations. The DC^{3}S pattern — reliability-conditioned uncertainty inflation followed by feasible-set projection — should therefore be read as a reusable systems idea for CPS settings such as HVAC dispatch, water treatment, and autonomous vehicle motion planning, wherever measurement uncertainty and hard safety constraints meet.

The conclusion-level locked numeric summary is therefore explicit: 7.11% DE cost savings, 0.30% DE carbon reduction, 6.13% DE peak shaving, 0.11% US cost savings, 0.13% US carbon reduction, and 0.00% US peak shaving.

References

[1] V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic Learning in a Random World.* Springer, 2005.

[2] G. Shafer and V. Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9:371–421, 2008.

[3] Y. Romano, E. Patterson, and E. Candès. Conformalized quantile regression. *Advances in Neural Information Processing Systems*, 32, 2019.

[4] I. Gibbs and E. Candès. Adaptive conformal inference under distribution shift. *Advances in Neural Information Processing Systems*, 34, 2021.

[5] M. Zaffran, O. Féron, Y. Goude, J. Josse, and A. Dieuleveut. Adaptive conformal predictions for time series. *International Conference on Machine Learning*, 2022.

[6] R. F. Barber, E. J. Candès, A. Ramdas, and R. J. Tibshirani. Conformal prediction beyond exchangeability. *Annals of Statistics*, 51(2):816–845, 2023.

[7] D. Bertsimas, D. B. Brown, and C. Caramanis. Theory and applications of robust optimization. *SIAM Review*, 53(3):464–501, 2011.

[8] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski. *Robust Optimization*. Princeton University Press, 2009.

[9] J. R. Birge and F. Louveaux. *Introduction to Stochastic Programming*. Springer, 2nd edition, 2011.

[10] R. T. Rockafellar and S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–42, 2000.

[11] D. Rosewater, R. Baldick, and S. Santoso. Risk-averse model predictive control design for battery energy storage systems. *IEEE Transactions on Smart Grid*, 11(3):2014–2022, 2020.

[12] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu. Safe reinforcement learning via shielding. *AAAI Conference on Artificial Intelligence*, 2018.

[13] O. Bastani. Safe reinforcement learning with nonlinear dynamics via model predictive shielding. *American Control Conference*, 2021.

[14] B. Könighofer, F. Lorber, N. Jansen, and R. Bloem. Shield synthesis for reinforcement learning. *Leveraging Applications of Formal Methods*, 2020.

[15] L. Sha. Using simplicity to control complexity. *IEEE Software*, 18(4):20–28, 2001.

[16] M. Leucker and C. Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.

[17] S. M. Dibaji, M. Pirani, D. B. Flamholz, A. M. Annaswamy, K. H. Johansson, and A. Chakrabortty. A systems and control perspective of CPS security. *Annual Reviews in Control*, 47:394–411, 2019.

[18] V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.

[19] T. Chen, Y. Jin, H. Lv, A. Yang, M. Liu, B. Chen, Y. Xie, and Q. Chen. Applications of lithium-ion batteries in grid-scale energy storage systems. *Transactions of Tianjin University*, 26:208–217, 2020.

[20] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and H. Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[21] V. Borisov, T. Leemann, K. Sessler, J. Haug, M. Pawelczyk, and G. Kasneci. Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[22] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 2017.

A Replication Checklist

All main claims should be reproducible from the following steps:

```
# Validate paper claims
python3 scripts/validate_paper_claims.py
python3 scripts/sync_paper_assets.py --check

# Train models and build UQ artifacts
make train-all-ba

# Run CPSBench fault sweep
make r1-cpsbench

# Ablation study and publication checks
make r1-full
make r1-verify
make paper-sync
```

Canonical run IDs: DE `20260217_165756`, US `20260217_182305`. Dashboard lock: `2026-02-17T11:15:38`. All artifacts are hash-tracked in the release manifest.

B Hyperparameters and Configuration

Table 24: Core training and RAC-Cert hyperparameters.

group	key	value	source
train_forecast_de	horizon_hours	24	configs/train_forecast.yaml
train_forecast_de	lookback_hours	168	configs/train_forecast.yaml
train_forecast_de	cross_validation.n_folds	10	configs/train_forecast.yaml
train_forecast_de	cross_validation.gap	24	configs/train_forecast.yaml
train_forecast_us	horizon_hours	24	configs/train_forecast_eia930.yaml
train_forecast_us	lookback_hours	168	configs/train_forecast_eia930.yaml
train_forecast_us	cross_validation.n_folds	10	configs/train_forecast_eia930.yaml
train_forecast_us	cross_validation.gap	24	configs/train_forecast_eia930.yaml
dc3s_rac	sensitivity_probe	heuristic	configs/dc3s.yaml
dc3s_rac	sens_eps_mw	25.0	configs/dc3s.yaml
dc3s_rac	sens_norm_ref	0.5	configs/dc3s.yaml
dc3s_rac	beta_reliability	0.7	configs/dc3s.yaml
dc3s_rac	beta_sensitivity	0.5	configs/dc3s.yaml
dc3s_rac	k_sensitivity	0.4	configs/dc3s.yaml
regime_cqr	enabled	True	configs/uncertainty.yaml
regime_cqr	n_bins	3	configs/uncertainty.yaml
regime_cqr	vol_window	24	configs/uncertainty.yaml
regime_cqr	quantile_backend_policy	strict	configs/uncertainty.yaml

Key configuration values from the locked `configs/` directory:

- Forecast: 24-hour horizon, 168-hour lookback, 10-fold time-aware CV with 24-hour gap.
- Conformal: $\alpha = 0.10$, volatility binning window = 168 hours.
- RAC-Cert: $\kappa_r = 0.2$ (quality), $\kappa_d = 0.0$ (drift, disabled), $\kappa_s = 0.4$ (sensitivity), `infl_max` = 2.0, shrinkage $\tau = 30.0$.

- Reliability: $\lambda_{\text{delay}} = 0.002$, $\beta_{\text{spike}} = 0.25$, $\gamma_{\text{ooo}} = 0.35$, $w_{\min} = 0.05$.
- Drift detection: Page-Hinkley with $\delta = 0.01$, $\lambda = 5.0$, warmup 48 steps, cooldown 24 steps.
- Shield: projection mode, reserve SOC under drift = 8%.

C Data Assets and Feature Design

The feature design follows a leakage-safe temporal framing: all lag and rolling features use strictly past data relative to the forecast origin, with a 24-hour gap between training and test folds. Weather features include temperature, humidity, precipitation, cloud cover, and wind speed at multiple heights. Calendar features encode hour-of-day, day-of-week, month, weekend indicator, and season. The lag structure covers 1-hour, 24-hour, and 168-hour (weekly) lags for each target, plus rolling mean and standard deviation at 24-hour and 168-hour windows.

The US feature set is larger (114 vs. 94) primarily because the US pipeline includes additional lagged exogenous variables (price, carbon intensity at multiple lags) and more granular weather features from the expanded geographic coverage of three balancing authorities.

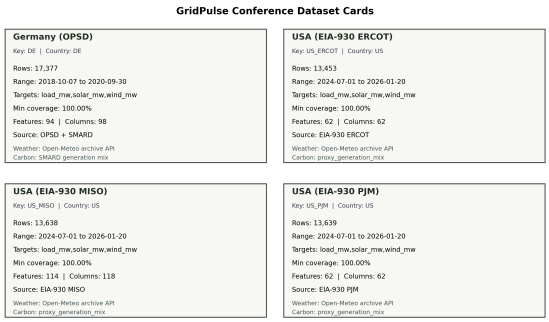

Figure 48: Dataset cards for all regions in the release scope.

D Additional Figures

Missing figure token: FIG15$_{R}OLLING_{B}ACKTEST_{R}MSE$

Figure 49: Rolling backtest RMSE across the evaluation horizon. Walk-forward windows (168-hour steps) show forecast accuracy evolution over time, with seasonal degradation visible in summer months.

Missing figure token: FIG55$_{M}ULTI_{H}ORIZON_{B}ACKTEST$

Figure 50: Multi-horizon backtest results. Forecast accuracy degrades gracefully with increasing horizon, supporting the 24-hour operational dispatch window.

Missing figure token: FIG16$_{E}RROR_{S}EASONALITY$

Figure 51: Forecast error seasonality patterns across targets. Solar errors peak in transition seasons (spring/autumn) due to variable cloud cover; load errors peak during extreme temperature episodes.

E Artifact Inventory

F Operational Failure Modes

G Notation and Symbols

H Extended Algorithm Details

This appendix provides the full algorithmic specification for two key DC^{3}S subsystems that are summarized in the main text: the FTIT state update and the reliability scoring function.

Algorithm 2 FTIT state update at each dispatch step

Require: Fault flags $\{f_t^{(k)}\}_{k=1}^{5}$, previous state $(n_t, s_t, \sigma_t^2, e_t)$, config $(\gamma_d, \alpha_k, \gamma_{\min}, \gamma_{\max}, \gamma_{\text{pow}})$
Ensure: Updated state $(n_{t+1}, s_{t+1}, w_t^{\text{FTIT}}, \sigma_{t+1}^2, e_{t+1}, \text{soc_tube})$

$n_{t+1} \leftarrow \gamma_d \cdot n_t + 1$
for each fault type k **do**
  $s_{t+1}^{(k)} \leftarrow \gamma_d \cdot s_t^{(k)} + \mathbb{1}[f_t^{(k)}]$
  $p_{t+1}^{(k)} \leftarrow s_{t+1}^{(k)} / \max(n_{t+1}, \epsilon)$
end for
$w_t^{\text{FTIT}} \leftarrow \prod_k [\text{clip}(1 - p_{t+1}^{(k)}, \epsilon, 1)]^{\alpha_k}$
$\sigma_{t+1}^2 \leftarrow \max(\sigma_{\text{floor}}^2,\ \gamma_d \cdot \sigma_t^2 + (1 - \gamma_d) \cdot \sigma_{\text{obs}}^2)$ ▷ residual variance
$\gamma_{\text{MW}} \leftarrow \gamma_{\min} + (1 - w_t^{\text{FTIT}})^{\gamma_{\text{pow}}} \cdot (\gamma_{\max} - \gamma_{\min})$
$e_{t+1} \leftarrow \text{clip}(\gamma_d \cdot e_t + \Delta t \cdot \gamma_{\text{MW}},\ e_{\min},\ e_{\max})$
$\text{soc_tube} \leftarrow [\text{SOC}_{\min} + e_{t+1},\ \text{SOC}_{\max} - e_{t+1}]$
return $(n_{t+1}, s_{t+1}, w_t^{\text{FTIT}}, \sigma_{t+1}^2, e_{t+1}, \text{soc_tube})$

These algorithms are presented in a form that matches the actual implementation in `src/gridpulse/dc3s/ftit.py` and `src/gridpulse/dc3s/quality.py` respectively. The default parameter values correspond to the locked configuration in `configs/dc3s.yaml` (Table 5).

Table 25: Optimization configuration from `configs/optimization.yaml`.

Parameter	Value	Role
Objective weights		
Cost weight (λ_c)	1.0	Energy cost coefficient
Carbon weight (λ_g)	1.2	Carbon-weighted cost coefficient
Battery parameters		
Capacity	20,000 MWh	Rated energy capacity ($E_{\max}$)
Max power	5,000 MW	Charge/discharge power limit ($P_{\max}$)
Efficiency	0.92	Round-trip charge/discharge efficiency
Degradation cost	\$10/MWh	Cycle-aging penalty per throughput
Constraint penalties		
Curtailment	\$500/MW	Penalty for wasted renewable energy
Unmet load	\$10,000/MW	High penalty (should not trigger)
Peak	\$1/MW	Small peak-demand penalty
Robust dispatch		
Risk weight (worst-case)	0.35	DRO blend: 1.0 = pure worst-case
Min SOC	1,000 MWh	Minimum allowed SOC
Initial SOC	10,000 MWh	Starting battery state (50%)

Missing figure token: FIG39$_{D}ISPATCH_{C}OMPARE$

Figure 52: Dispatch comparison across controller types during a representative week. DC^{3}S matches the deterministic controller during healthy telemetry and diverges only during degraded periods.

Table 26: Feature family composition by region.

Feature family	DE (count)	US (count)
Weather	40	56
Lag	36	42
Calendar	6	6
Total	**94**	**114**

Table 27: Target distribution snapshot (DE).

Target	Mean	Std	Min	Max	Median
Load (MW)	55,156	9,998	31,923	76,925	54,752
Wind (MW)	14,369	10,322	136	46,064	11,728
Solar (MW)	5,013	7,666	0	32,947	156

Table 28: Target distribution snapshot (US/MISO).

Target	Mean	Std	Min	Max	Median
Load (MW)	75,757	11,819	52,940	120,343	73,564
Wind (MW)	10,956	6,042	0	26,132	10,184
Solar (MW)	2,875	3,843	0	14,315	110

Missing figure token: FIG53$_{C}ASE_{S}TUDY_{D}ISPATCH$

Figure 53: Case study dispatch: a 48-hour window showing charge/discharge decisions, price signals, and SOC evolution under three controllers.

Missing figure token: FIG54$_{C}ASE_{S}TUDY_{W}EEK$

Figure 54: Case study week: a seven-day operational trace showing the interaction between forecast quality, reliability scoring, and dispatch behavior.

Missing figure token: FIG52$_{C}OST_{C}ARBON_{D}ETAIL$

Figure 55: Detailed cost–carbon tradeoff analysis. Each Pareto point represents a different weight configuration in the multi-objective optimizer.

Missing figure token: FIG56$_{U}SA_{D}ATA_{Q}UALITY$

Figure 56: US data quality deep dive. Coverage, missing-data patterns, and signal reliability across MISO, PJM, and ERCOT balancing authorities.

Missing figure token: FIG57$_{G}AP_{H}OURLY_{P}ROFILES$

Figure 57: DE–US hourly profile gap analysis. Structural differences in diurnal load, wind, and solar patterns between German and US grid regimes.

Missing figure token: FIG58$_{G}AP_{D}ISTRIBUTION$

Figure 58: DE–US distribution comparison. Target variable distributions differ in scale, skewness, and zero-inflation (especially solar), motivating region-specific calibration.

Table 29: Core manuscript and evidence artifacts.

Artifact	Purpose
`paper/PAPER_DRAFT.md`	Canonical thesis manuscript source
`paper/paper.tex`	Thesis LaTeX twin of the markdown source
`paper/paper_r1.tex`	Shorter conference derivative
`paper/metrics_manifest.json`	Canonical metric lock and display contract
`paper/claim_matrix.csv`	Claim-level provenance and status tracking
`scripts/validate_paper_claims.py`	Cross-file claim validation
`scripts/sync_paper_assets.py`	Paper-asset synchronization audit
`reports/impact_summary.csv`	Locked DE impact values
`reports/eia930/impact_summary.csv`	Locked US impact values
`reports/research_metrics_de.csv`	DE stochastic summary rows
`reports/research_metrics_us.csv`	US stochastic summary rows
`data/dashboard/manifest.json`	Profile lock metadata
`configs/dc3s.yaml`	DC^{3}S configuration (Table 5)
`configs/optimization.yaml`	Optimization parameters (Table 25)

Table 30: Certificate schema and audit-chain fields implemented by the DC^{3}S runtime.

Field group	Meaning	Audit / safety role
`command_id`, `certificate_id`, `created_at`	Unique dispatch event identity and timestamp	Orders events and anchors step-level traceability
`prev_hash`, `certificate_hash`	Hash-chain linkage across certificates	Makes the audit trail tamper-evident and sequential
`model_hash`, `config_hash`	Hashes of model artifacts and active configuration	Pins each dispatch step to the exact model/config state
`proposed_action`, `safe_action`, `dispatch_plan`	Optimizer output, repaired action, and plan context	Records what the optimizer wanted and what the shield allowed
`reliability`, `drift`, `reliability_w`, `inflation`, `uncertainty`	Telemetry-quality and interval-inflation context	Explains why conservatism changed at that step
`intervened`, `intervention_reason`	Whether repair or fallback occurred and why	Supports intervention-rate analysis and incident diagnosis
`guarantee_checks_passed`, `guarantee_fail_reasons`	Post-repair deterministic invariant results	Verifies that the emitted action satisfies runtime safety checks
`gamma_mw`, `e_t_mwh`, `soc_tube_lower_mwh`, `soc_tube_upper_mwh`	FTIT tube width and tightened SOC envelope	Captures the reliability-conditioned safety margin actually enforced
`true_soc_violation_after_apply`, `assumptions_version`	Post-apply evaluation flag and assumption contract version	Separates runtime decision logging from evaluation semantics

Table 31: Operational failure modes and safety controls.

Failure Mode	Trigger	Detection	Mitigation	Fallback
Missing/stale data	Upstream delay	Readiness checks	Block unsafe dispatch	Hold current state
Forecast drift	Distribution shift	KS/model monitors	Retraining workflow	Conservative mode
Solver infeasible	Extreme intervals	Solver status flags	Safe infeasible response	Grid-only baseline
Unsafe command	SOC/power violation	BMS validation	Reject with detail	Maintain safe state
Control degradation	Heartbeat timeout	Watchdog detection	Lock remote control	Manual/local
Certificate failure	DuckDB write error	Audit path check	Conservative mode	Disable dispatch

Table 32: Principal notation used throughout this paper.

Symbol	Definition
State and telemetry	
$\mathbf{z}_t$	Clean (true) telemetry vector at time t
$\tilde{\mathbf{z}}_t$	Degraded (observed) telemetry after $\mathcal{D}$
$\mathcal{D}$	Degradation operator (dropout, delay, spikes, reordering)
$\mathrm{SOC}_t^{\mathrm{true}}$	True state of charge (MWh)
$\mathrm{SOC}_t^{\mathrm{obs}}$	Observed state of charge (MWh)
Battery	
$E_{\max}$	Rated battery energy capacity (MWh)
$P_{\max}$	Maximum charge/discharge power (MW)
η_c, η_d	Charge and discharge efficiency
$R_{\max}$	Maximum ramp rate (MW/step)
a_t	Dispatch action at step t (MW, positive = discharge)
$\mathcal{A}_t$	Feasible action set at step t
Forecasting and uncertainty	
$\hat{C}_t(\alpha)$	Base conformal prediction interval at level α
$\hat{C}_t^{\mathrm{RAC}}(\alpha)$	RAC-Cert inflated interval
α	Nominal miscoverage level (default 0.10)
$\hat{q}$	Empirical conformal quantile
DC^{3}S parameters	
w_t	Telemetry reliability score $\in [w_{\min}, 1]$
d_t	Drift detection flag (binary)
s_t	Normalized sensitivity signal $\in [0, 1]$
$\kappa_r, \kappa_d, \kappa_s$	Inflation coefficients for quality, drift, sensitivity
$\varepsilon_{\mathrm{sens}}$	Sensitivity probe perturbation (MW)
τ_{shrink}	Shrinkage time constant (steps)
Objective and economics	
$\lambda_c, \lambda_g, \lambda_p, \lambda_d$	Cost, carbon, peak, and degradation weights
c_t, g_t	Electricity price and carbon intensity at step t
L_t	Net load (load minus renewables) at step t
VSS	Value of the stochastic solution
EVPI	Expected value of perfect information
FTIT state	
γ_d	Exponential decay factor for fault rates (0.98)
$p_t^{(k)}$	Rolling empirical fault rate for fault k
γ_{MW}	SOC tube width (MW)
e_t	Accumulated SOC margin (MWh)
σ_t^2	Exponentially weighted residual variance

Algorithm 3 Reliability scoring from raw telemetry

Require: Telemetry event $\tilde{\mathbf{z}}_t$, previous event $\tilde{\mathbf{z}}_{t-1}$, cadence Δ_{exp}

Ensure: Reliability score w_t, component penalties

$p_{\text{miss}} \leftarrow \max(0, 1 - \text{frac_missing}(\tilde{\mathbf{z}}_t))$

$\delta_s \leftarrow |t_{\text{arrive}} - t_{\text{expected}}|$ ▷ staleness in seconds

$p_{\text{delay}} \leftarrow \exp(-\lambda_{\text{delay}} \cdot \max(0, \delta_s))$ ▷ $\lambda = 0.002$

$p_{\text{ooo}} \leftarrow \begin{cases} 1 - \gamma_{\text{ooo}} & \text{if out-of-order detected} \\ 1 & \text{otherwise} \end{cases}$ ▷ $\gamma = 0.35$

$r_{\text{spike}} \leftarrow \max_j |\tilde{z}_{t,j} - \tilde{z}_{t-1,j}| / (|\tilde{z}_{t-1,j}| + \epsilon)$ ▷ relative jump

$p_{\text{spike}} \leftarrow \begin{cases} 1 & \text{if } r_{\text{spike}} \leq \beta_{\text{spike}} \\ 1/(1 + r_{\text{spike}} - \beta_{\text{spike}}) & \text{otherwise} \end{cases}$ ▷ $\beta = 0.25$

$w_{\text{raw}} \leftarrow p_{\text{miss}} \cdot p_{\text{delay}} \cdot p_{\text{ooo}} \cdot p_{\text{spike}}$

$w_t \leftarrow \max(w_{\min}, \min(1, w_{\text{raw}}))$ ▷ $w_{\min} = 0.05$

return w_t, $(p_{\text{miss}}, p_{\text{delay}}, p_{\text{ooo}}, p_{\text{spike}})$